

Intelligent Surveillance and Identification for Secure Campus

A Project Report Submitted

in Partial Fulfillment of the Requirements

for the Degree of

Bachelor of Technology

(Computer Science & Engineering)

Submitted By

Abu Reeyan (Reg. No. : 222025109276)

Bittu Kumar (Reg. No. : 222025109271)

Sushma Kumari (Reg. No. : 222025109261)

Kishun Kumar (Reg. No. : 222025109246)

Under Guidance of

Dr. Narendra Kumar



Faculty of Computing and Information Technology

Usha Martin University, Ranchi

November 20, 2025

BONAFIDE CERTIFICATE

Certified that this project report titled **“Intelligent Surveillance and Identification for Secure Campus”** is the bonafide work of Abu Reeyan, Bittu Kumar, Sushma Kumari and Kishun Kumar, who carried out the research under my supervision. Certified further, that to the best of my knowledge the work reported herein does not form part of any other project report or dissertation on the basis of which a degree or award was conferred on an earlier occasion on this or any other candidate.

(Signature of the concerned Supervisor of the Organization with Organization Seal)

(Certificate to be countersigned by the HOD.)

DECLARATION

I do hereby declare that this report entitled **“Intelligent Surveillance and Identification for Secure Campus”**, submitted by Abu Reeyan, bearing Reg. No. : 222025109276, Bittu Kumar, bearing Reg. No. : 222025109271, Sushma Kumari, bearing Reg. No. : 222025109261, Kishun Kumar, bearing Reg. No. : 222025109246, in the fulfillment of the requirement for the degree of Bachelor of Technology in Computer Science & Engineering to Usha Martin University, Ranchi, is my own and it is not submitted to any other institute.

Abu Reeyan

Reg. No. : 222025109276

Bittu Kumar

Reg. No. : 222025109271

Sushma Kumari

Reg. No. : 222025109261

Kishun Kumar

Reg. No. : 222025109246

CERTIFICATE

This is to certify that entitled **“Intelligent Surveillance and Identification for Secure Campus”** being submitted by Abu Reeyan, bearing Reg. No. : 222025109276, Bittu Kumar, bearing Reg. No. : 222025109271, Sushma Kumari, bearing Reg. No. : 222025109261, Kishun Kumar, bearing Reg. No. : 222025109246, in the fulfillment of the requirement for the degree of Bachelor of Technology in Computer Science & Engineering to Usha Martin University, Ranchi, is a bonafide work carried out under my supervision. The matter embodied in this report is original and has not been submitted for the award of any other degree.

Dr. Sharmistha Roy

HOD

Dr. Narendra Kumar

Guide

ACKNOWLEDGEMENTS

I express my deepest sense of gratitude to my guide **Dr. Narendra Kumar** Faculty of Computing and Information Technology, Usha Martin University, Ranchi, for suggesting the subject of work and constant supervision throughout this work. His co-operation and timely suggestions have been unparalleled stimuli for me to travel eventually towards the completion of this project report. Indeed her continuous involvement has helped me in bringing of this project work which otherwise would have remained a distant dream.

I am indeed thankful to **Dr. Sharmistha Roy**, HOD, Faculty of Computing and Information Technology, Usha Martin University, Ranchi, for giving me permission to carry out my project work. I would like to express my gratitude to all teaching and non-teaching staff members of Faculty of Computing and Information Technology, Usha Martin University, Ranchi, for their co-operation in my work.

Abu Reeyan

Reg. No. : 222025109276

Bittu Kumar

Reg. No. : 222025109271

Sushma Kumari

Reg. No. : 222025109261

Kishun Kumar

Reg. No. : 222025109246

Abstract

Ensuring campus security and student compliance with institutional uniform policies has become increasingly challenging with manual monitoring methods. This project proposes an AI-powered system that utilizes computer vision and deep learning to identify and classify students in real-time based on their uniform color and facial features. The system is designed to analyze live video feeds from CCTV cameras, detect student faces using the DeepFace model, and retrieve their information from a connected database. Simultaneously, it applies color segmentation techniques to determine the uniform color and maps it to the respective department. A comparison is made between the department obtained from the uniform and the one retrieved from the database to verify student identity and detect uniform violations or unauthorized individuals. The methodology includes video acquisition, frame preprocessing, face recognition, color-based uniform detection, and integration with a querying database. The implementation demonstrates significant improvements in accuracy and efficiency over existing image-based systems, particularly in video-based applications. The integration with a live database makes real-time tracking and data retrieval possible, transforming security protocols across educational institutions. The findings show that the proposed system is effective, scalable, and suitable for real-world deployment. It sets the foundation for future developments such as automated attendance, behavioral monitoring, and campus-wide surveillance integration. Overall, this project demonstrates a practical and intelligent approach to student monitoring through AI and deep learning.

Keywords – Campus, Classification, Cybersecurity, Detection, Department, DeepFace, Surveillance, Vision, Security.

TABLE OF CONTENTS

S.no.	Topic Name	Page No.
1.	Bonafide	ii
2.	Declaration.....	iii
3.	Certificate.....	iv
4.	Acknowledgement.....	v
5.	Abstract	vi
6.	List of Figures.....	ix
7.	List of Tables.....	x
8.	List of Abbreviations.....	xi

Chapter	Topic Name	Page no.
1.	Introduction.....	1-5
	1.1 Overview	1
	1.2 Background.....	2
	1.3 Applications & Features of the Project.....	3-4
	1.4 Objective	4
	1.5 Problem Statement.....	4-5
	1.6 Research Gap.....	5
2.	Literature Review	6-12
	Table of Literature Review.....	10-12
3.	Requirements Analysis.....	13-22
	3.1 System Requirement.....	13-14
	3.2 Hardware Required.....	15
	3.3 Software Required.....	15
	3.4 ER Diagram.....	16

	3.5 Use Case Diagram.....	17
	3.6 Data Flow Diagram.....	18-21
4.	Proposed Methodology.....	22-27
	4.1 Methodology.....	22-24
	4.2 Block Diagram.....	25-27
5.	Model Testing.....	28-33
6.	Result Analysis.....	34-37
	5.1 Output images of the Project.....	35-36
	5.2 Confusion Matrix.....	36-37
7.	Conclusion & Future Scopes.....	38-40
	6.1 Conclusion.....	38
	6.2 Future Scope	39
	6.3 Limitations.....	40
	References.....	41-42
	Appendix.....	43-76

List of Figures

S. No.	Figure No.	Figure Name	Page No.
1	3.4	ER Diagram of Intelligent Surveillance and Identification for Secure Campus.....	16
2	3.5	Use Case Diagram of Intelligent Surveillance and Identification for Secure Campus.....	18
3	3.6.1	Level 0 Data Flow Diagram of Intelligent Surveillance and Identification for Secure Campus.....	20
4	3.6.2	Level 1 Data Flow Diagram of Intelligent Surveillance and Identification for Secure Campus.....	21
5	3.6.3	Level 2 Data Flow Diagram of Intelligent Surveillance and Identification for Secure Campus.....	22
6	4.1	Block Diagram of Intelligent Surveillance and Identification for Secure Campus.....	25
7	5.1	Bounding Box Classes of the Dataset of the Intelligent Surveillance and Identification for Secure Campus.....	32
8	5.2	Graph of the Loss Function of the Intelligent Surveillance and Identification for Secure Campus.....	33
9	5.3	Validation Images of the Intelligent Surveillance and Identification for Secure Campus.....	34
10	6.1	Output Images of Intelligent Surveillance and Identification for Secure Campus.....	35
11	6.2	Confusion Matrix of the trained model of the Intelligent Surveillance and Identification for Secure Campus	36

List of Tables

S. No.	Table No.	Table Name	Page No.
1	2.1	Table of Literature Review.....	10-12
2	5.1	Table of Dataset Distribution.....	28
3	5.2	Test Case Table.....	29

List of Abbreviations

Abbreviations	Full Form
CNN	Convolutional Neural Network
CCTV	Closed Circuit Television
DCNN	Deep Convolutional Neural Network
AI	Artificial Intelligence
HSV	Hue, Saturation and Value
API	Application Programming Interface
ROI	Region of Interest
FPS	Frames Per Second
UI	User Interface
ID	Identification
CV	Computer Vision
KNN	K-Nearest Neighbors
SQL	Structured Query Language
RGB	Red, Green, Blue
TC	Test Case

Chapter 1

Introduction

1.1 Overview

Ensuring student safety, verifying identities, and enforcing institutional discipline are critical responsibilities in today's educational institutions. With the exponential rise in student enrollment and the continuous expansion of campus infrastructure, universities and colleges face growing difficulties in maintaining effective monitoring and security across their premises. Traditionally, these responsibilities have been addressed through manual methods: security personnel stationed at entry points, administrative staff visually verifying students, and physical enforcement of dress codes. However, these approaches are rapidly becoming inadequate in the face of increasing student populations and decentralized campus layouts. Manual surveillance is labour-intensive, inconsistent, and prone to human error. Security personnel can overlook violations due to fatigue, distraction, or limited field of view, while administrative staff may find it difficult to accurately verify student identity and dress code compliance, especially during peak hours. As a result, unauthorized access, uniform violations, and identity misrepresentation can go unnoticed, posing potential risks to campus safety and decorum. In this context, educational institutions need smarter, more reliable, and automated solutions to support or replace manual intervention with more accurate and scalable systems. In most academic institutions, students are assigned specific uniforms based on their departments, making uniform color a simple yet effective visual cue for department-level classification. However, checking for compliance manually across hundreds or thousands of students is inefficient and subjective. At the same time, identity checks through ID cards is often bypassed or manipulated. These problems underscore the urgent need for intelligent systems that can monitor students automatically, verify their identity in real-time, and detect violations proactively without disrupting campus operations. The rapid advancement of Artificial Intelligence, Computer Vision, and Deep Learning technologies presents a transformative opportunity to address these challenges. AI-powered surveillance systems can now detect faces, recognize individuals, and classify clothing attributes like color and texture using trained models with a high degree of accuracy. These capabilities, when integrated with CCTV infrastructure and institutional databases, form the foundation of a real-time, automated, and intelligent student identification and verification system.

1.2 Background

The advancement of education in the 21st century has not only transformed classroom learning but also the way educational institutions manage infrastructure, safety, and discipline. With the rapid growth in student intake, expansion of academic departments, and development of sprawling campus environments, universities and colleges today face new and evolving challenges. Among these, campus security and uniform discipline enforcement are two of the most pressing and foundational responsibilities for administration and security departments. Historically, institutions have relied on manual surveillance, such as physical security personnel monitoring entry points, patrolling premises, and reviewing closed-circuit television (CCTV) footage. While these methods served their purpose in smaller or less crowded campuses, they have become increasingly inefficient and insufficient in large, modern educational institutions. As student populations rise and campuses become more decentralized, traditional methods struggle to scale up. Manually observing hundreds or thousands of students to ensure uniform compliance or identify intruders is labour-intensive, time-consuming, and highly prone to human error. In most educational settings, especially in India and other countries with large student bases, students are assigned uniforms based on their academic department. This system provides a visual cue that helps both faculty and security personnel identify a student's academic affiliation. While the use of uniforms promotes equality and discipline, enforcing dress code compliance manually, especially in large institutions with multiple entry and exit points, is extremely challenging. Security staff may overlook violations, and there is no consistent, automated way to flag uniform mismatches. Moreover, manual verification of student identities through ID cards or physical registers is not only inefficient but also vulnerable to manipulation, proxy attendance, and impersonation. This gap in effective monitoring and verification creates serious vulnerabilities not just in discipline enforcement, but in overall campus security. Unidentified or unauthorized individuals might enter the campus, dress code violations might go unchecked, and false identity claims may go unnoticed. These issues underscore the need for automated, intelligent systems that can operate consistently, accurately, and autonomously across different parts of the campus. In recent years, Artificial Intelligence (AI) and Computer Vision have emerged as transformative technologies in the field of surveillance and automation. The ability of AI systems to detect faces, recognize identities, and process visual inputs such as clothing colors and patterns has improved dramatically.

1.3 Applications & Features of the Project

The Intelligent Surveillance and Identification System for Secure Campus is designed to serve as a practical, scalable, and intelligent solution for enhancing safety, monitoring, and administrative efficiency in educational institutions. The system's application goes beyond mere observation it actively verifies identity, enforces policy, and maintains digital records, making it suitable for a wide range of real-world academic use cases.

1. **Campus Entry Monitoring:** This system is highly effective at all main entry and exit points of the institution. As students walk through monitored areas, the system instantly captures their image, performs facial recognition, and verifies whether the individual is an enrolled student. If the face is not recognized or is found to be unauthorized, the system immediately triggers an alert, helping prevent intrusions and potential security threats.

2. **Uniform Compliance Enforcement:** One of the most common challenges in academic institutions is ensuring students adhere to their department-specific uniform guidelines. This system detects and analyses the uniform color of each student and compares it to their registered department. Any mismatch is flagged, allowing the institution to take quick disciplinary action and ensuring a consistent and professional academic environment.

3. **Departmental Classification:** Beyond identity, the system can classify students in real time based on the uniform color, which represents their department. This capability can be used for managing department-restricted zones (e.g., labs, workshops) where only authorized students should be granted access. It enhances security by reducing the risk of cross-departmental impersonation or unauthorized movement.

4. **Attendance Assistance:** With further integration, the system can be enhanced to automate student attendance by logging entry and exit timestamps when a student is recognized. This removes the need for manual attendance marking in classrooms or lecture halls, improves accuracy, and saves time for both students and faculty.

5. **Real-Time Alerts for Violations:** When any irregularity is detected such as an unrecognized face, uniform mismatch, or attempted access to a restricted area the system generates real-time alerts. These alerts are instantly pushed to an admin dashboard and can also be integrated with SMS, email, or mobile app notifications for faster response by the security team.

6. Administrative Logging: Each event, whether a successful match or violation, is logged with details such as student name, ID, time, location, and a snapshot. These logs serve as audit trails for administrative reviews and disciplinary committees. Reports can be generated weekly or monthly to analyze trends, policy violations, or security breaches.

1.4 Objective

The primary objective of this project is to:

- Provide smart surveillance by integrating DeepFace for facial recognition and color segmentation for uniform detection in real-time CCTV streams.
- Automate student identification by matching facial features with a centralized institutional database, removing the need for ID cards or manual attendance.
- Ensure departmental uniform compliance by detecting uniform colors and comparing them with the expected dress codes for each student's department.
- Generate real-time alerts for unidentified faces, uniform mismatches, or unauthorized activities through the admin dashboard and future notification channels.
- Offer a user-friendly web-based dashboard that displays real-time activities, violations, and logs for effective monitoring and decision-making.
- Reduce human surveillance workload by automating identification and uniform verification tasks, improving operational efficiency.

1.5 Problem Statement

In the current era of digital transformation, educational institutions are facing increasing pressure to enhance campus safety, maintain institutional discipline, and ensure operational efficiency. As universities and colleges continue to expand in size, infrastructure, and student population, the complexity of campus security and student management grows exponentially. Institutions now operate across multi-block campuses, with multiple entry and exit points, separate academic departments, and a high degree of student mobility. In this environment, the limitations of traditional, manual surveillance systems have become starkly evident. Conventional surveillance practices such as deploying guards at gates, checking ID cards manually, visually confirming uniforms, and observing CCTV monitors in real time are both inefficient and unsustainable. These methods are entirely dependent on human judgment, which is prone to fatigue, distraction, and error. In high-traffic environments, where

thousands of students pass through checkpoints daily, it's nearly impossible for human observers to maintain consistent attention, especially when verifying identity or uniform compliance. This leads to gaps in monitoring, missed violations, unauthorized access, and unreliable attendance tracking. Another major shortcoming of traditional systems is the lack of automated verification mechanisms. Student identity verification often relies on physical ID cards, which can be forgotten, lost, or misused. Proxy attendance, impersonation, and the inability to detect unauthorized individuals are persistent problems. Moreover, the physical act of checking IDs and verifying uniforms consumes significant administrative time and resources, reducing institutional efficiency. Uniform compliance is a crucial aspect of discipline in many institutions, especially where each academic department is assigned a specific uniform color. This system is designed to visually distinguish students based on their field of study and maintain order. Institutions aiming to become “smart campuses” must adopt systems that go beyond passive monitoring. There is a growing need for a robust, intelligent, and real-time surveillance system that can automatically detect student faces with high accuracy, identify uniform color and validate it against department standards, match live inputs with backend student records, trigger immediate alerts in case of mismatches or unidentified individuals, and maintain detailed logs for auditing, discipline, and reporting purposes.

1.6 Research Gap

- There is a lack of integrated systems that combine real-time facial recognition, uniform color detection, and database verification into a unified surveillance platform.
- Limited research exists on automated uniform compliance detection using color-based computer vision techniques in educational environments.
- Very few solutions support real-time monitoring and instant alerting for violations such as unauthorized access or mismatched uniforms.
- Current systems lack centralized architectures that unify CCTV feeds, facial data, uniform details, and institutional records for accurate cross-verification.
- Most surveillance models fail to handle real-world issues like lighting variations, occlusion, and crowding, reducing accuracy in dynamic campus environments.

Chapter 2

Literature Review

The implementation of intelligent surveillance systems using artificial intelligence and computer vision has gained considerable attention in recent years, particularly in enhancing public safety, automation, and monitoring efficiency. In the context of educational institutions, the fusion of real-time facial recognition, uniform detection, and centralized database integration is still evolving, and limited research has explored such systems holistically. This chapter summarizes relevant studies that form the foundation of the proposed system and highlights the gaps that this project aims to bridge.

1. **J. J. K, K. N. S, B. Vadivel, G. G and P. V, "Efficient YOLOv8 Model for Licence, Helmet and Person Detection," ICPCSN 2025, Salem, India, 2025, pp. 404-408, doi: 10.1109/ICPCSN65854.2025.11035282** : This paper presents an optimized YOLOv8-based model tailored for real-time security and surveillance tasks, including license plate, helmet, and person detection. By integrating SPPF layers, depthwise separable convolutions, and a lightweight MobileNet backbone, the system achieves improved accuracy and computational efficiency compared to standard YOLOv8. The model demonstrates strong performance, making it suitable for deployment on small computing devices in practical surveillance environments.
2. **Z. Li and J. Wang, "Multi-person gesture recognition for complex environmental based on improved Yolov8 algorithm," IAECST 2025, Guangzhou, China, 2025, pp. 614-619, doi: 10.1109/IAECST60924.2025.10502852** : This study enhances the YOLOv8 algorithm by integrating the Global Attention Mechanism (GAM), GSConv, and Slim Neck design, forming the YOSRLO model for multi-person gesture recognition in complex environments. The model addresses challenges such as small target detection and diverse gesture patterns, achieving high accuracy and improved recognition rates. Experimental comparisons show that YOSRLO outperforms standard YOLOv8, making it effective for gesture recognition in real-world settings.
3. **J. Nandre, S. Rai and B. R. Kanawade, "Comparative Analysis of Transfer Learning CNN for Face Recognition," CONIT 2024, Hubli, India, 2024, pp. 1-6, doi: 10.1109/CONIT55038.2024.9847946** : This research evaluates the performance of three popular CNN architectures—VGG16, ResNet-50, and MobileNet—for face recognition using

transfer learning techniques. The models are tested on both static images and webcam-based input using datasets from Kaggle and local sources. By analyzing training accuracy, validation accuracy, and loss metrics, the study identifies performance differences among the architectures, offering insights for real-time face recognition applications such as surveillance and attendance systems.

4. D. S. Parker, "Integrating AI and DBMS through stream processing," ICDE 2024, Los Angeles, USA, 2024, pp. 259-260, doi: 10.1109/ICDE.2024.47224 : This paper discusses a novel approach to bridging the gap between AI systems and database management systems by using a unified stream-processing model. The Tangram project, which integrates Prolog with relational DBMS, demonstrates how extending Prolog into a functional language (Log(F)) simplifies data processing and aligns both AI and DBMS workflows. The integration method offers an elegant and efficient solution to longstanding compatibility challenges between these technologies.

5. T. Babu, E. P. M and R. R. Nair, "Real-Time Facial Expression Recognition Using Deep Learning for Enhanced Human-Machine Interactions," BITCON 2024, Dhanbad, India, 2024, pp. 1-6, doi: 10.1109/BITCON63716.2024.10984634 : This work develops a real-time facial expression recognition system using deep learning techniques, trained on the FER-2013 dataset to classify seven basic emotions. The enhanced CNN model outperforms standard benchmarks by incorporating data augmentation and additional layers. A user-friendly GUI and real-time detection capability make the system applicable in areas like mental health monitoring, security, marketing, and interactive technologies.

6. H. R. Tohidypour, M. T. Pourazad and P. Nasiopoulos, "A Security-Centric Deep Learning Enabled Camera Solution for Real-Time Human Fall Detection," WiMob 2023, Thessaloniki, Greece, 2023, pp. 42-46, doi: 10.1109/WiMob55322.2023.9941359 : This paper introduces an embedded deep-learning-enabled fall detection system integrated into a camera device for real-time remote healthcare monitoring. Using TensorRT optimization, efficient memory allocation, and GPU-accelerated GStreamer processing, the system achieves 84.44% accuracy with low latency. The solution provides a secure, non-intrusive, and cost-effective method for continuous fall detection, making it suitable for elderly care and medical surveillance.

7. M. Doi, T. Matsumoto, A. Kimachi, S. Nishi and N. Ikoma, "Robust color object tracking method against illumination color Change," SCIS-ISIS 2023, Kitakyushu,

Japan, 2023, pp. 718-722, doi: 10.1109/SCIS-ISIS.2023.7044769 : This research proposes a color-object tracking model based on the Gray World Assumption (GWA) to handle illumination color changes effectively. By modeling object color relative to the average frame color and using particle filters for tracking, the method maintains strong robustness even under abrupt lighting variations. Experimental results confirm the feasibility and effectiveness of the proposed tracking approach.

8. Y. Zhao, H. Liu, Q. Wang and C. Li, "A Multi-Object Tracking Method Using IoU and Color Fusion Model," ICSIP 2023, Nanjing, China, 2023, pp. 747-751, doi: 10.1109/ICSIP61881.2023.10671489 : This paper presents a two-stage multi-object tracking framework that combines IoU similarity and color histogram features using a fusion model. YOLOX performs object detection, while Kalman filters predict object locations. The fusion of IoU and color similarity improves object association and reduces ID switches. The system achieves top-ranking performance on the MOT17 and MOT20 benchmarks, demonstrating high accuracy in complex tracking scenarios.

9. Y. Qi, Y. Wang and Y. Liu, "Object Tracking Based on Deep CNN Feature and Color Feature," ICSP 2022, Beijing, China, 2022, pp. 469-473, doi: 10.1109/ICSP.2022.8652470 : This study combines deep CNN features extracted using VGG-Face with color histogram features to create a robust object tracking model. An adaptive particle filter integrates these features for dynamic tracking. The system handles occlusion and appearance variations effectively, improving tracking performance in challenging visual environments.

10. Zhiyuan Han, Tingrong Xu and Zhaohua Chen, "An improved color-based tracking by particle filter," TMEE 2022, Changchun, China, 2022, pp. 2512-2515, doi: 10.1109/TMEE.2022.6199732 : This work improves particle filter-based object tracking by replacing Monte Carlo sampling with Quasi-Monte Carlo sampling, reducing sample correlation and improving accuracy. The approach enhances object modeling using robust color features and increases tracking speed and stability, outperforming traditional particle filter methods in simulations.

11. S. Pati, A. Majumder, M. Gangopadhyay, G. Sen and A. Chakraborty, "Advanced Attendance Monitoring: A Unified System Using QR Codes and Facial Recognition," IEMENTech 2021, Kolkata, India, 2021, pp. 1-4, doi: 10.1109/IEMENTech65115.2021.10959151 : This paper integrates QR codes with facial

recognition to create a modern attendance system that improves accuracy and reduces manual errors. Students scan their QR codes upon entry and exit, followed by face verification, ensuring identity authenticity. The system stores all data in a structured database, enabling automated and transparent attendance tracking across academic cycles.

12. R. Nidhya, D. J. A. Pabi, U. DivyaSree, B. Abhishek, A. Harika and S. A. Alim, "Exploring Facial Recognition Technologies for Classroom Management," ICCSP 2021, Melmaruvathur, India, 2021, pp. 1702-1707, doi: 10.1109/ICCSP60870.2021.10544143 :

This paper proposes a high-accuracy classroom monitoring system using facial recognition and machine learning. Student facial features are captured through segmentation and processed into structured arrays for training recognition models. Real-time person detection and detailed log generation enable automated attendance tracking. Achieving 99.38% accuracy, the system demonstrates strong potential for smart classroom management.

2.1 Table of Literature Review

Table 2.1 – Table of Literature Review

S. No.	Paper Title	Author(s)	Year	Methodology Used	Pros	Cons	Journal Name	Page No.	Volume No.	DOI
1	Efficient YOLOv8 Model for Licence, Helmet and Person Detection	J. J. K, K. N. S, B. Vadivel, G. G, P. V	2025	Optimized YOLOv8 using SPPF, depthwise convolutions, and MobileNet backbone for improved detection.	High accuracy and computational efficiency suitable for small devices.	Performance may depend on hardware accelerators for best results.	ICPCSN 2025	404–408	04	10.1109/ICPCSN 65854.2025.11035282
2	Multi-person gesture recognition for complex environmental based on improved Yolov8 algorithm	Z. Li, J. Wang	2025	Enhanced YOLOv8 with GAM, GSConv, Slim Neck and Real-ESRGAN segmentation to form YOSRLO model.	Achieves high accuracy and recognizes small gesture targets effectively.	Increased model complexity may lead to higher computational load.	IAECST 2025	614–619	12	10.1109/IAECST 60924.2025.10502852
3	Comparative Analysis of Transfer Learning CNN for Face Recognition	J. Nandre, S. Rai, B. R. Kanawade	2024	Used transfer learning with VGG16, ResNet-50, and MobileNet CNN models for face recognition.	Provides strong accuracy and flexibility using pre-trained deep CNNs.	Performance varies across architectures depending on dataset conditions.	CONIT 2024	1–6	01	10.1109/CONIT5 5038.2024.9847946
4	Integrating AI and DBMS through stream processing	D. S. Parker	2024	Retrofitted AI and DBMS into a common stream-processing model using Log(F) language.	Offers elegant and efficient integration of AI systems with DBMS.	Limited by computational capabilities of early AI systems and legacy DBMS.	ICDE 2024	259–260	7	10.1109/ICDE.2024.47224
5	Real-Time Facial Expression Recognition Using Deep Learning	T. Babu, E. P. M, R. R. Nair	2024	Built a CNN-based facial expression classifier trained on FER-2013 with enhanced preprocessing and GUI.	Demonstrates strong real-time performance and improved accuracy over benchmarks.	Accuracy may drop in low-light or occluded face conditions.	BITCON 2024	1–6	32	10.1109/BITCO N63716.2024.10984634

6	A Security-Centric Deep Learning Enabled Camera Solution for Real-Time Human Fall Detection	H. R. Tohidypour, M. T. Pourazad, P. Nasiopoulou	2023	Embedded deep-learning fall detection optimized with TensorRT and GPU-based processing.	Achieves real-time performance with reliable detection accuracy.	Hardware dependency may limit use in low-resource environments.	WiMob 2023	42–46	11	10.1109/WiMob55322.2023.9941359
7	Robust color object tracking method against illumination color change	M. Doi, T. Matsumoto, A. Kimachi, S. Nishi, N. Ikoma	2023	Utilized Gray World Assumption and particle filtering to handle illumination color shifts.	Maintains robust tracking even under sudden illumination changes.	Effectiveness decreases in scenes where GWA assumptions fail.	SCIS-ISIS 2023	718–722	5	10.1109/SCIS-ISIS.2023.7044769
8	A Multi-Object Tracking Method Using IoU and Color Fusion Model	Y. Zhao, H. Liu, Q. Wang, C. Li	2023	Combined IoU similarity with color histogram features in a two-stage association tracking framework.	Achieves top ranking MOTA accuracy on MOT17 and MOT20 datasets.	Fusion computation increases processing time in crowded scenes.	ICSIP 2023	747–751	12	10.1109/ICSIP61881.2023.10671489
9	Object Tracking Based on Deep CNN Feature and Color Feature	Y. Qi, Y. Wang, Y. Liu	2022	Integrated VGG-Face deep features with color histograms using adaptive particle filtering.	Handles occlusion and appearance variation effectively.	Heavy CNN feature extraction increases computation cost.	ICSP 2022	469–473	11	10.1109/ICSP.2022.8652470
10	An improved color-based tracking by particle filter	Zhiyuan Han, Tingrong Xu, Zhaohua Chen	2022	Replaced Monte Carlo sampling with Quasi-Monte Carlo sampling in particle filtering.	Improves tracking accuracy and speed by reducing sampling correlation.	May struggle with complex motion patterns beyond color-based modeling.	TMEE 2022	2512–2515	9	10.1109/TMEE.2022.6199732
11	Advanced Attendance Monitoring: A Unified System Using QR Codes and Facial Recognition	S. Pati, A. Majumder, M. Gangopadhyay, G. Sen, A. Chakraborty	2021	Integrated QR code authentication with facial recognition for automated attendance tracking.	Enhances accuracy and reduces human errors in institutional attendance.	System may face reliability issues under poor lighting or QR misreads.	IEMEN Tech 2021	1–4	21	10.1109/IEMEN Tech65115.2021.10959151

12	Exploring Facial Recognition Technologies for Classroom Management	R. Nidhya, D. J. A. Pabi, U. DivyaSree, B. Abhishek, A. Harika, S. A. Alim	2021	Used machine learning and OpenCV-based segmentation to build high-accuracy classroom monitoring.	Achieves 99.38% accuracy and automates attendance with real-time detection.	Requires consistent camera alignment and quality for reliable recognition.	ICCSP 2021	1702–1707	37	10.1109/ICCSP60870.2021.10544143
----	--	--	------	--	---	--	------------	-----------	----	----------------------------------

Chapter 3

Requirements Analysis

3.1 System Requirements

To develop and deploy the Intelligent Surveillance and Identification System, specific hardware, software, and network components are required to ensure high accuracy, real-time monitoring, and reliable identity verification. These requirements are designed to support continuous video streaming, color-based uniform classification, face detection, facial embedding generation, and identity matching. Proper selection of tools and technologies ensures efficient processing, secure data handling, and seamless operation across all modules within a dynamic campus environment.

i) Functional Requirements

a) High-Quality Video Capture Module

The system must be capable of capturing high-resolution video streams from CCTV cameras installed across the campus. It should support stable frame rates, adaptive lighting adjustments, and uninterrupted streaming for real-time processing.

b) Uniform Color Detection Module

The system must detect and classify student uniform colors using computer vision algorithms. It should identify distinct uniform shades accurately and categorize them according to department or designated groups.

c) Facial Detection and Embedding Generation

The system must detect student faces from video feeds using the DeepFace model and generate corresponding facial embeddings. It should work effectively in varying lighting conditions and crowded environments.

d) Identity Matching and Verification

The system must match detected facial embeddings with records stored in the institutional database to verify student identity. It should fetch and display associated information such as student name, roll number, and department.

e) Real-time Identity Display on Dashboard

The system must display real-time identity details and department information on the admin dashboard, ensuring security personnel can immediately recognize students and detect unauthorized individuals.

ii) Non-Functional Requirements

a) Real-Time Performance and Low Latency

The system must maintain real-time performance with minimal latency during video processing, face detection, and identity recognition. Delays in processing should be minimized to support live surveillance operations.

b) Scalability

The system must be scalable enough to process multiple CCTV streams simultaneously. It should accommodate an increasing number of cameras and higher data loads as the campus expands without performance degradation.

c) Accuracy and Reliability

The system must maintain high accuracy in uniform color detection and face recognition to avoid misclassification. It should ensure consistent performance across diverse lighting conditions, angles, and camera qualities.

d) Ease of Deployment and Maintainability

The system must be easy to install, configure, and maintain within existing campus infrastructure. Regular updates and bug fixes should be implementable without affecting ongoing surveillance operations.

e) Usability and Interface Quality

The admin dashboard must be user-friendly, intuitive, and responsive. It should allow security personnel to easily view live streams, identify individuals, review logs, and manage system configurations.

3.2 Hardware Required

Hardware Component	Specification
CPU	AMD Ryzen 5 or higher
RAM	Minimum 8 GB
Storage	Minimum 10 GB free disk space
GPU	NVIDIA GTX 1650 or higher
Network	Ethernet/Wi-Fi
CCTV	IP-based HD camera (20 FPS)

3.3 Software Required

Software Component	Specification
Operating System	Ubuntu 24.04 LTS
Programming Language	Python 3.12
Database	MySQL
Libraries	OpenCV, DeepFace, FastAPI
Development Tools	VS Code
Version Control	Git

3.4 ER Diagram

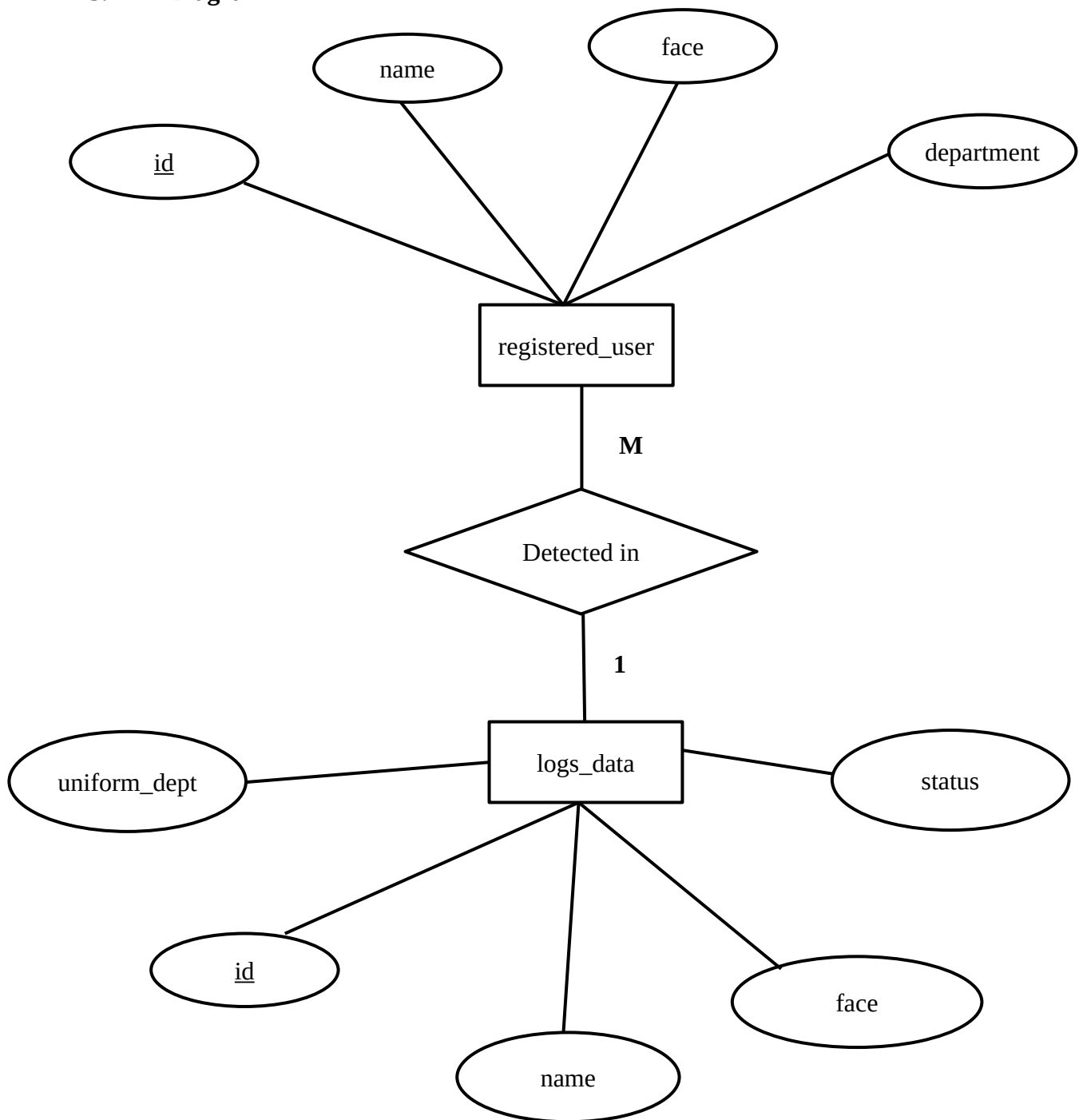


Fig-3.4 : ER Diagram of Intelligent Surveillance and Identification for Secure Campus

3.5 Use Case Diagram

The use case diagram represents the functional behavior of the “Intelligent Surveillance and Identification System” and shows how external actors interact with its automated monitoring workflow. The process begins with the User, who starts the surveillance by performing the Start Detection use case, which activates the system’s internal detection pipeline. Once initiated, the system automatically moves to the Capture Frames use case, which receives live video frames from the Camera actor, representing the source. These captured frames are then passed to two included recognition modules. The first is Face Recognition, which communicates with the DeepFace actor an external facial recognition service that analyzes facial features and returns identity information. The second is Uniform Recognition, which works with the Shirt Detector Model actor, responsible for detecting uniform colors or attributes linked to departments. Results from both recognition paths flow into the Cross Verification use case, where the system checks whether the identified person matches the detected uniform or department. An extend relationship links Cross Verification to Generate Alert, meaning an alert is triggered only if a mismatch or anomaly is found. The Generate Alert use case then notifies the User about suspicious activity.

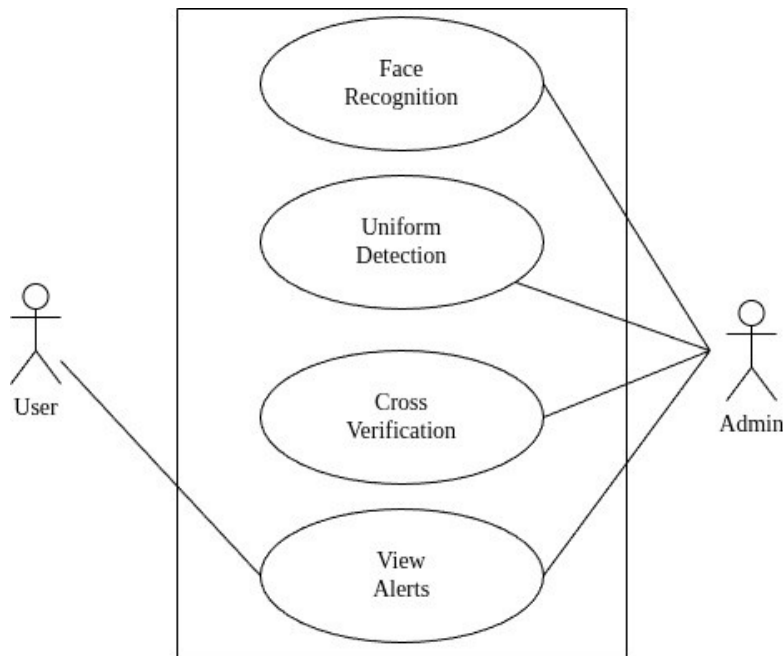


Fig-3.5: Use Case Diagram of the Intelligent Surveillance and Identification for Secure Campus

3.6 Data Flow Diagram

The Data Flow Diagram (DFD) illustrates the complete movement of data within the Intelligent Surveillance and Identification System, showing how different components interact to deliver continuous, automated monitoring. At the highest level, the system receives input from several external sources. The User acts as the initiator by starting the surveillance process or configuring system parameters. Simultaneously, the CCTV camera provides a continuous stream of live video, serving as the primary data source. Additionally, external AI recognition models, such as the DeepFace facial identification service and a uniform detection model, supply specialized processing capabilities that the system relies on for accurate classification. Once these inputs enter the system, they flow into the core Surveillance Module, where the video stream is broken down into individual frames for real-time processing. Each frame undergoes two parallel operations: facial feature extraction and uniform attribute detection. The extracted data is then forwarded to the respective AI services, which return identity information and department or uniform classification results. After receiving these outputs, the system performs a cross-verification step, comparing the detected identity with the corresponding uniform or department to confirm whether the individual is compliant. If the system detects a mismatch such as a student wearing the wrong uniform or an unrecognized individual in a restricted area it triggers an alert and immediately notifies the user, ensuring timely intervention. Regardless of whether the outcome is normal or suspicious, all results are stored in the Detection Details Store, capturing details such as timestamps, recognized individuals, uniform type, and verification outcomes. This storage enables the user to later retrieve logs, review detection histories, and generate detailed reports, ensuring transparency, accountability, and traceability. When no mismatch is detected, the event is simply logged as valid, and the surveillance process continues without interruption. Overall, the DFD clearly demonstrates how external entities, recognition modules, processing components, and data storage mechanisms collaborate to maintain continuous, intelligent, and automated surveillance. It provides a structured visualization of data flow, helping stakeholders understand system logic, identify dependencies, and ensure that operations remain efficient, accurate, and real time.

Level 0 DFD

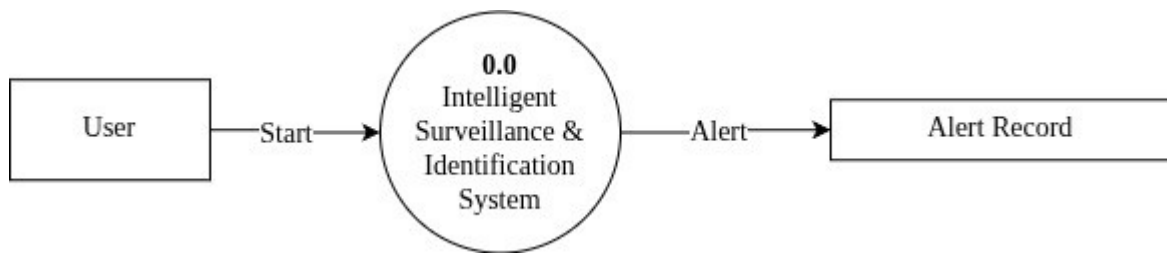


Fig-3.6.1 : Level 0 Data Flow Diagram Intelligent Surveillance and Identification for Secure Campus

This Level-0 Data Flow Diagram, also known as the context diagram, provides a high-level overview of how the Intelligent Surveillance System interacts with external entities and manages data flow. At this topmost level, the system is represented as a single unified process without revealing its internal components or detailed operations. The diagram highlights two key external elements: the User and the Alert Record data store. The interaction begins with the User, who triggers the system by sending a Start request. This action activates the surveillance process, signaling the system to begin monitoring and analysis. At this stage, the diagram does not show how footage is captured or how detection is performed; instead, it emphasizes that the system's internal logic is encapsulated within one central process. Once the surveillance process is running, the Intelligent Surveillance System continuously evaluates incoming data to detect any suspicious or non-compliant activity. If the system identifies a potential threat, anomaly, or irregular behavior, it generates an Alert as its primary output. This alert represents a critical notification indicating that an event of interest has occurred and may require attention or review. The generated alert is then sent to the Alert Record data store, where it is logged and preserved for future reference. Storing alerts ensures that the system maintains a historical record of detected incidents, enabling traceability, reporting, audits, and post-event investigation. The simplicity of this diagram is intentional; Level-0 DFDs are meant to show only the most essential data interactions without complexity. It clearly illustrates the core input-process-output relationship: the user initiates the system, the system processes surveillance activities internally, and the system outputs alerts that are stored externally. This high-level perspective helps stakeholders understand the system's boundaries, the primary data exchanged, and the purpose of the system without diving into technical details.

Level 1 DFD

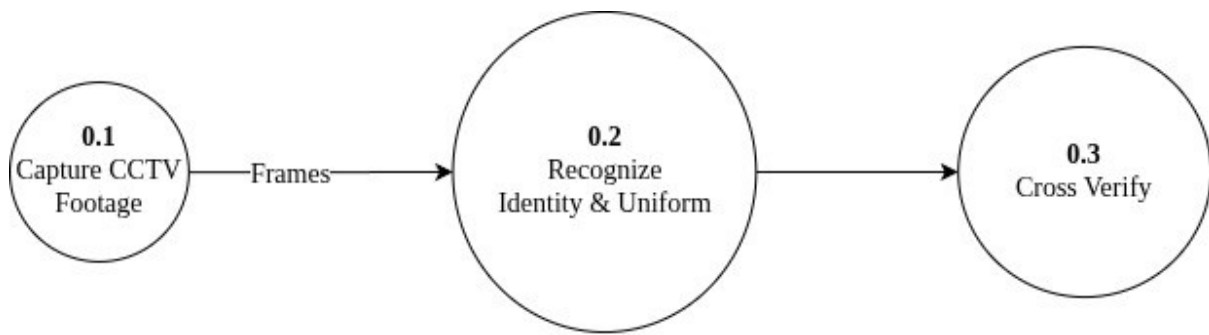


Fig-3.6.2 : Level 1 Data Flow Diagram Intelligent Surveillance and Identification for Secure Campus

This Level-1 Data Flow Diagram provides a clearer breakdown of how the Intelligent Surveillance and Identification System operates once the user initiates the process. The workflow begins with the User, who sends a Start request to activate the system. This request is directed to the first major process, Capture CCTV Footage, which continuously receives and extracts video frames from the live camera feed. These frames serve as the primary raw input for the system's analysis. The captured frames are then passed to the second process, Recognize Identity & Uniform. At this stage, the system performs two key recognition tasks: identifying the person's face to determine their identity and analyzing uniform attributes to identify their department or group. Instead of treating these as separate low-level processes, this diagram groups them into one combined recognition function suitable for Level-1 representation. The recognition results are forwarded to the Cross Verify process, where the system checks whether the detected identity matches the associated uniform or department. If a mismatch or suspicious condition is found, the system generates an Alert. This alert is then stored in the Alert Record data store, ensuring that all flagged events are logged for future review and reporting. The system continues monitoring until stopped by the user, highlighting an ongoing real-time surveillance cycle.

Level 2 DFD

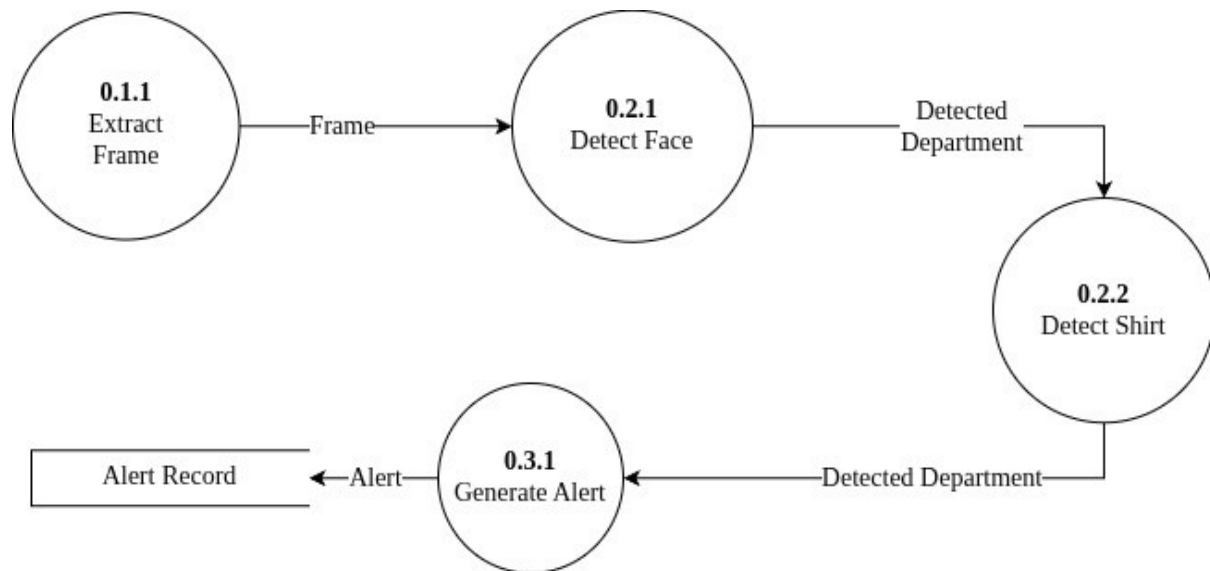


Fig-3.6.3 : Level 2 Data Flow Diagram Intelligent Surveillance and Identification for Secure Campus

The Level-2 breakdown of the surveillance workflow, showing how the system processes visual data step-by-step after the user initiates monitoring. The process begins with the User, who sends a Start request, activating the system. This request flows into the Capture CCTV Footage process, which continuously receives live video and extracts individual Frames for analysis. These frames are then passed to the Detect Face process, where the system identifies the person in the frame and determines their identity or associated department. The output of this stage, labeled Detected Department, is forwarded to the Detect Shirt process. Here, the system analyzes the uniform or shirt color to determine the department or group the person visually appears to belong to. Both pieces of information the department inferred from the face and the department inferred from the uniform are sent to the Cross Verify process. This step checks whether the detected identity aligns with the detected uniform. If a mismatch or suspicious case is found, the system generates an Alert, indicating a potential violation or anomaly. The alert is then stored in the Alert Record data store, ensuring that all flagged events are logged for future review, tracking, or reporting. This structure highlights a logical, layered flow of detection and verification.

Chapter 4

Proposed Methodology

4.1 Methodology

The methodology of this project is centered around integrating Deep Learning and Computer Vision techniques to enable automated student identification and uniform verification in real time. The system is designed to process video feeds, detect and recognize faces and shirts, identify department from color of the detected shirt, and match results with student records stored in a backend database. The implementation involves the following key stages and algorithms:

1. Video Acquisition and Frame Preprocessing

The system begins by acquiring continuous video streams from CCTV cameras placed across the campus, ensuring that every frame contains sufficient detail for analysis. From this live feed, frames are periodically extracted and passed through a series of preprocessing steps designed to enhance image quality and standardize input data. These steps include resizing frames to a consistent resolution, normalizing pixel values, reducing noise artifacts using smoothing filters, and converting the color space from RGB to HSV, which significantly improves the reliability of color-based segmentation tasks. This preprocessing pipeline ensures that each frame is optimally prepared for both face detection and uniform analysis in downstream modules.

2. Facial Recognition Using DeepFace

Facial recognition is accomplished through the DeepFace model, an advanced deep learning framework designed specifically for robust and accurate face identification. The model uses a specialized convolutional neural network architecture that generates 128-dimensional facial embeddings for each detected face. These embeddings serve as high-level, discriminative feature representations unique to each individual. To verify identity, the system compares the embedding of the detected face with stored embeddings retrieved from a secured student database, using distance metrics such as cosine similarity and Euclidean distance. If the similarity score is within the accepted threshold, the model confirms the match and retrieves the student's profile, including name and registered department. This integration between

DeepFace and the backend database allows the system to instantly associate facial features with institutional records, enabling precise identity verification.

3. Department Identification

Department identification is handled through a two-stage process involving shirt detection followed by color extraction. First, a custom object detection model trained on 17,000 annotated images with labeled shirt bounding boxes is used to locate shirt regions in each frame. The architecture of this model is built on a deep convolutional neural network that includes a strong backbone for deep feature extraction, a multi-scale neck component using SPPF and PAFPN to fuse spatial information, and a detection head that outputs bounding boxes and classification scores. It is optimized using Binary Cross-Entropy for classification accuracy and Complete Intersection over Union loss for precise bounding box regression. Once the shirt region is detected and isolated, the system applies K-Means clustering to the cropped area to determine the dominant color in HSV space. This extracted color is then matched against a predefined mapping of colors associated with each academic department, enabling the system to automatically infer the student's department based solely on uniform color.

4. Cross Verification

To ensure accuracy and eliminate inconsistencies, the system performs a cross-verification step that compares the department obtained from the shirt color with the department retrieved during facial recognition. If both data sources match, the system confirms successful verification and marks the student as compliant. In cases where the uniform color suggests a different department than the one stored in the database, the system immediately flags this as a uniform violation, indicating a potential misuse of uniforms or unauthorized movement across department-restricted zones. Additionally, if the facial recognition module fails to identify the person in the frame, the system automatically labels the individual as an unauthorized entity, prompting security personnel to review the anomaly. This layered verification mechanism ensures that both appearance and identity are consistently validated.

5. Alert Generation and Admin Dashboard

The final stage of the methodology involves generating alerts and displaying all processed information on an interactive admin dashboard built using FastAPI. The dashboard provides

real-time updates on verified students, their identity information, detected departments, and any violations encountered. Unauthorized entries, mismatches between shirt color and department, and unidentified individuals are prominently highlighted to enable quick decision-making by administrators. The combined use of a custom-trained object detection model, DeepFace for facial recognition, and K-Means clustering for color analysis ensures that the system achieves high accuracy, operational speed, and modularity qualities essential for maintaining an efficient, intelligent, and trustworthy campus surveillance environment.

4.2 Block Diagram

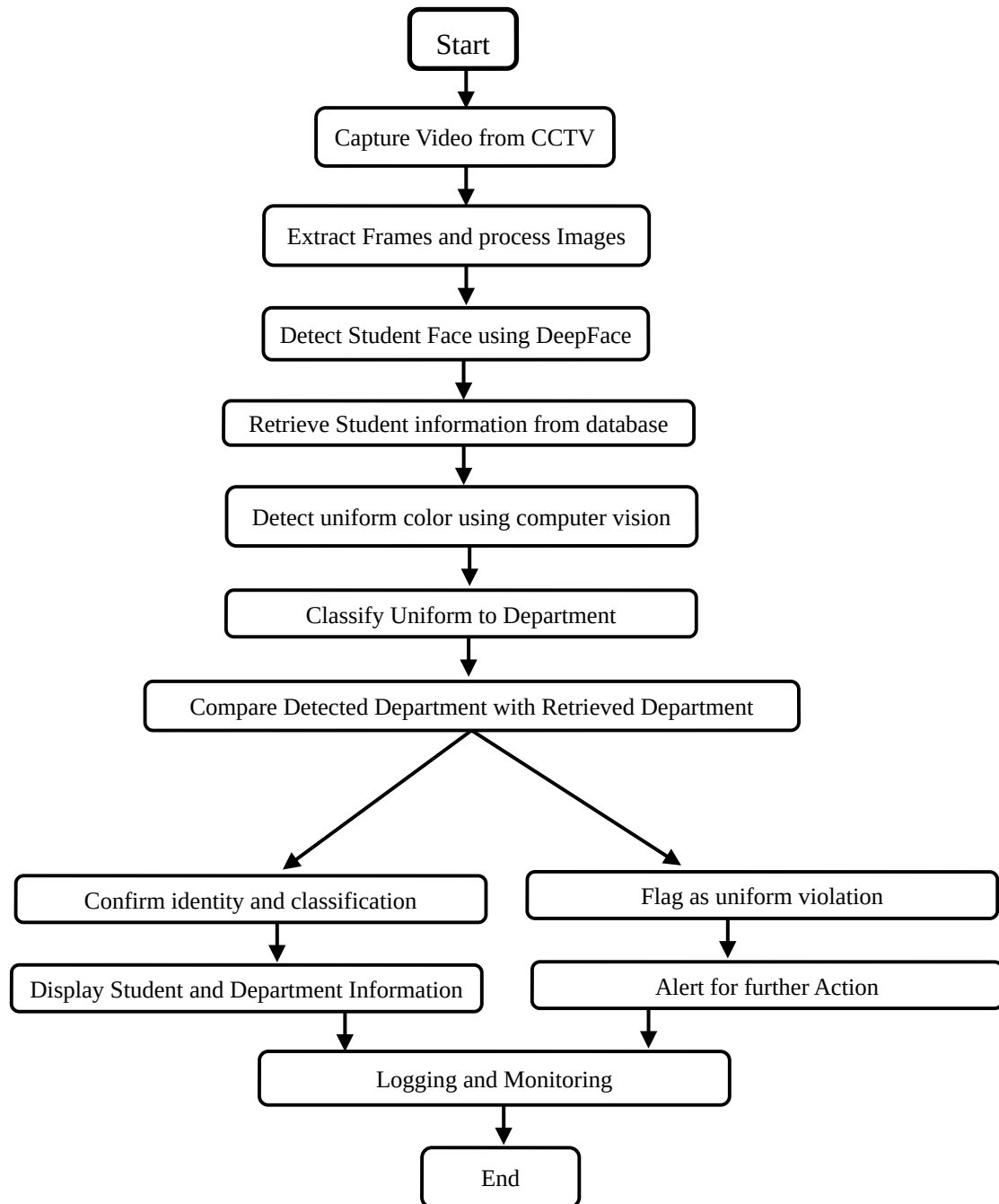


Fig – 4.1: Block Diagram of the Intelligent Surveillance and Identification System

Explanation

1. CCTV Camera: The process begins with CCTV cameras strategically installed across the campus to monitor student activity. These cameras operate continuously, capturing live video streams as students enter buildings, walk through corridors, or move across outdoor areas. Their primary role is to provide uninterrupted real-time surveillance data that forms the foundation for the system's automated monitoring and analysis.

2. Capture Video: The captured live footage is accessed by the system's video intake module, which fetches the raw video feed directly from the camera network. This feed may either be streamed for immediate processing or buffered temporarily depending on the system's configuration. The goal of this stage is to ensure smooth and uninterrupted transmission of video frames to the subsequent processing pipeline.

3. Extract Frames: Since real-time video consists of numerous frames per second, the system extracts individual frames at predefined intervals. These static frames are easier and faster to analyze using computer vision algorithms. Frame extraction prevents redundant processing by ensuring that only meaningful images are analyzed, enabling efficient monitoring while maintaining continuous coverage across different camera angles.

4. Verify Identity: In this phase, the extracted frames are processed using the DeepFace recognition model to detect the student's face and verify their identity. Once a match is confirmed, the system queries the student database to fetch authenticated records such as the student's name, roll number, department, and registered facial image. This step ensures that the detected individual is accurately identified based on institutional records.

5. Detect Uniform Color: Each frame is then scanned for uniform color using color segmentation and dominant color detection techniques. The system focuses on the clothing region, isolating the most prominent color present in the student's shirt. Based on predefined color-to-department mappings, the detected color is translated into a department classification for cross-verification.

6. Database: The database serves as the central repository holding verified student records, including personal details, department assignments, and authorized face images. It ensures reliable identity retrieval for the facial recognition module and supports comparison operations during the verification stage. Maintaining accurate and updated data in this storage is critical for system performance and correctness.

7. Classify Department: Using the uniform color detected in the previous step, the system assigns a probable department to the student. This classification is derived from predefined rules mapping each department to a specific uniform color code. The output of this step represents the department inferred visually from the student's attire.

8. Match Department: The system then compares the department retrieved through facial recognition with the department inferred through uniform color detection. This comparison is essential for ensuring that students comply with the institutional dress code and are wearing the correct department-specific attire. A match indicates normal behavior, while a mismatch indicates a potential violation.

9. Flag Violation: If the two departments do not align, the system identifies the event as a uniform violation or suspicious anomaly. This step is crucial for enforcing discipline and ensuring that unauthorized or non-compliant behavior is detected early.

10. Generate Alert: Upon detecting a violation, the system automatically generates an alert containing relevant information such as the student's identity, timestamp, camera location, and type of violation. This alert is forwarded to the campus administration or security team so they can take timely action.

11. Display Student Info: Finally, the verified student information—including name, identity, and cross-verified department—is displayed on a centralized dashboard. Security personnel can monitor real-time events, view alerts, and validate uniform compliance visually, ensuring quick response and effective campus surveillance.

Chapter 5

Model Testing

The shirt detection model underwent a comprehensive training and evaluation process to ensure its reliability in real-world scenarios. The custom dataset used for model development initially consisted of 7,351 manually annotated images, each containing bounding boxes marking shirt regions. Through extensive preprocessing and augmentation techniques such as rotation, flipping, scaling, brightness adjustment, and noise injection the dataset was significantly expanded to 17,069 images, providing broader diversity and reducing the risk of overfitting. The model was trained for 25 full epochs, during which it learned to identify shirt patterns, shapes, and color distributions across various lighting conditions, backgrounds, and student postures. After the training phase was completed, the model was rigorously evaluated to determine its capability to generalize to unseen data. This evaluation involved both quantitative analysis using a dedicated validation split and qualitative inspection through real-time video frames and still images captured from operational environments. To ensure structured and reliable testing, the dataset was divided into training, validation, and testing subsets. The distribution is shown below:

Table 5.1: Table of Dataset Distribution of Intelligent Surveillance and Identification System

Dataset	Images	In Percentage (%)
Train Set	14577	85%
Valid Set	1343	8%
Test Set	1149	7%

This balanced partitioning ensured that the model was primarily exposed to a large training set for effective learning, while still reserving adequate data for unbiased validation and final testing. The validation set helped fine-tune hyperparameters, monitor training performance, and detect signs of underfitting or overfitting. Meanwhile, the test set provided a final performance measure after all training adjustments were completed, ensuring that reported results truly reflected the model's ability to generalize. The sufficiently large and diverse dataset played a crucial role in achieving strong model performance. The extensive

augmentation process introduced variations in angle, lighting, and clothing appearance, enabling the model to learn features that were not overly sensitive to specific patterns in the original images. As a result, despite being trained for only 25 epochs, the model demonstrated strong generalization capability and maintained enough detection accuracy on previously unseen images. This indicates that the model effectively captured the underlying visual characteristics associated with shirts, rather than memorizing the training data.

Table 5.2: Test Case Table of Intelligent Surveillance and Identification System

TC No.	Test Case Description	Input	Expected Output	Actual Output	Status
TC-01	Test face detection using DeepFace	Live camera frame containing a visible human face	Face detected and bounding box drawn	As expected	Pass
TC-02	Test face embedding generation	Detected face region	128-dimensional facial embedding generated	As expected	Pass
TC-03	Face recognition with matching record	Face of a registered student	Student name, ID, and department retrieved	Correct identity details displayed	Pass
TC-04	Face recognition with unregistered face	Unknown person	Output: "Unknown" identity	System displayed "Unknown"	Pass
TC-05	Shirt detection using custom model	Frame containing full upper body	Shirt bounding box detected	Shirt correctly localized	Pass
TC-06	Shirt detection at long distance	Subject > defined max distance	Shirt detection skipped	System ignored far shirt region	Pass

[Test Case No: TC-01] - Verify Face Detection Using DeepFace

“Verify Face Detection Using DeepFace” ensures that the system can accurately identify and localize human faces in a live CCTV frame. When a person enters the camera’s field of view, the model should automatically scan the frame and detect the presence of a face without requiring manual intervention. The detection module must draw a bounding box around the identified face, confirming that the camera feed is being processed correctly. This test validates whether the face detection model performs consistently across different lighting conditions, viewing angles, and distances. If the face is clearly visible, the system must reliably detect it and trigger the next stage of the pipeline embedding generation ensuring real-time responsiveness and uninterrupted surveillance operations.

[Test Case No: TC-02] - Verify Facial Embedding Generation

“Verify Facial Embedding Generation” evaluates whether the system can convert a detected face into a numerical embedding vector using the DeepFace framework. After a bounding box is successfully drawn, the system must extract the face region and generate a 128-dimensional feature embedding that uniquely represents the person. This embedding is essential for accurate identity verification and matching with the institutional database. The test ensures that embeddings are generated consistently even with variations in facial expressions, partial shadows, or minor occlusions. A successful outcome confirms that the model has processed the facial features correctly and produced an embedding ready for comparison, forming the foundation of reliable identity recognition.

[Test Case No: TC-03] - Verify Face Recognition With Matching Student Record

“Verify Face Recognition With Matching Student Record” ensures that the system can correctly identify a registered student by comparing their facial embedding with stored embeddings in the institutional database. When a known student appears in front of the camera, the system should match their embedding with the correct database record using similarity thresholds. Upon successful matching, the system must display accurate identity details such as the student’s name, roll number, and department on the dashboard. This test validates the correctness of the recognition algorithm, checks the integrity of stored embeddings, and confirms that the retrieval of student information is both accurate and instantaneous, enabling real-time identity verification within the surveillance environment.

[Test Case No: TC-04] - Verify Recognition of Unregistered or Unknown Face

“Verify Recognition of Unregistered or Unknown Face” checks whether the system can differentiate between known registered students and unidentified individuals. When a person whose face embedding does not exist in the database appears, the system must correctly label the identity as “Unknown.” This prevents false identification and ensures that unauthorized individuals are appropriately flagged for review. The test confirms that the decision threshold for similarity is properly configured so that the system neither misidentifies an outsider as a student nor incorrectly classifies a student as unknown. A correct outcome of this test strengthens the security reliability of the entire surveillance framework.

[Test Case No: TC-05] - Verify Shirt Detection Using Custom Model

“Verify Shirt Detection Using Custom Model” validates whether the trained shirt detection model can accurately locate and draw bounding boxes around a student’s shirt region in a CCTV frame. When a person is visible, the object detection module should identify the upper-body clothing area based on features learned from the dataset. The system must correctly position the bounding box even when the student is slightly angled or partially shifted in the frame. This test ensures that the detection head, backbone, and neck components of the model function cohesively, producing stable detections essential for further color extraction. Successful detection confirms the model’s readiness to perform uniform color analysis and department classification.

[Test Case No: TC-06] : Verify Shirt Detection at Long Distance

“Verify Shirt Detection at Long Distance” ensures that the system behaves correctly when the subject stands beyond the optimal detection range. Uniform detection is distance-dependent, and the model should ignore frames where the shirt area is too small or unclear to analyze reliably. When the person stands farther than the predefined maximum distance, the system must intentionally skip detection rather than producing inaccurate or noisy outputs. This test confirms that distance validation logic is functioning as expected, preventing false detections, incorrect color extraction, and misclassification. Proper handling of distance constraints improves system stability in real-world CCTV deployment where subjects frequently vary in their proximity to the camera.

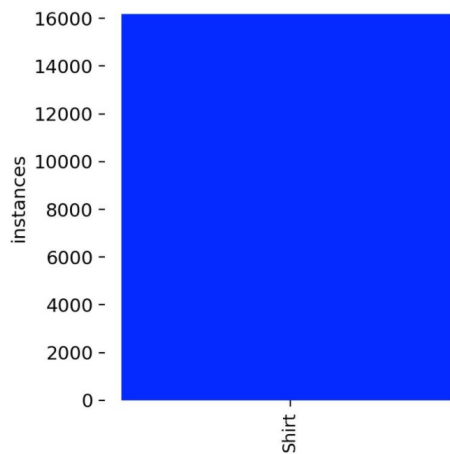


Fig-5.1 : Bounding Box Classes of the Shirt Detection Dataset of Intelligent Surveillance and Identification for Secure Campus

This visualization summarizes the annotation distribution for the "Shirt" class. The plot shows a high instance count, with approximately 16,000 labeled shirts. The top-right image displays the bounding box outlines, revealing that most shirts are detected near the image center with consistent sizing. The heatmap shows that the x and y coordinates of shirt detections are concentrated around the center, indicating a typical vertical alignment. The bottom-right plot illustrates a strong correlation between bounding box width and height, suggesting that shirts generally maintain a consistent aspect ratio and scale proportionally across the dataset.

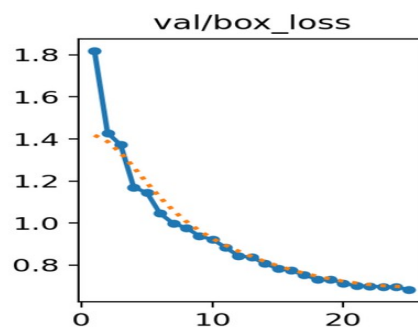


Fig-5.2 : Graph of the Loss Function of Intelligent Surveillance and Identification for Secure Campus

Throughout the training, there was a clear downward trend in the box loss, classification loss, and distribution focal loss, suggesting that the model was effectively learning to minimize detection errors. The steady increase in mAP and recall values across epochs further validates that the model was learning meaningful features from the limited data. The graph shows the

validation box loss decreasing steadily over 25 epochs, indicating that the model is progressively improving its ability to predict accurate bounding boxes for shirts. The loss starts high at around 1.8, drops sharply in the early epochs, and then declines more gradually as the model refines its predictions. By the final epochs, the loss stabilizes near 0.75, showing good convergence and strong generalization without signs of overfitting. Overall, the curve reflects a healthy and effective training process for the shirt detection model.



Fig-5.3: Validation Images of Intelligent Surveillance and Identification for Secure Campus

For quantitative testing, the model's performance was evaluated using standard object detection metrics. In addition to metrics, visual inspections were conducted by running the model on test images. The predicted bounding boxes were visually compared to the expected locations of shirts. This helped verify that the model correctly identifies and localizes clothing item in various scenarios, such as different lighting conditions and background complexities. Qualitative testing, supported by visual results under result analysis, showed that the model could reliably detect shirts and pants in well-lit and unobstructed images. However, it occasionally misidentified clothing in cluttered backgrounds or under challenging lighting conditions. Overall, the model testing phase confirmed that the trained model is effective at detecting shirts and performs reliably across different scenarios. Future improvements could include longer training and hyperparameter tuning.

Chapter 6

Result Analysis

The system provides buttons to start and stop the camera. As the camera starts it instantly initiates the Face recognition and student identification process, as the student is identified, it displays the details of the student on the screen and moves to the next step.

In the second step the system starts detecting shirt of the person standing in front of the camera. After that it extracts the dominant color of the shirt and predicts the department of the matching shirt color.

Note: For the system to accurately detect the shirt, the person must maintain a distance from the camera. If the area of the shirt covers more than 30% of the camera screen, it won't be able to detect the shirt. This is because the CCTV cameras are placed at a distance from the path, so to simulate the same we have defined a min and max distance required for the shirt to be captured otherwise ignored.

In the third step, the department got from the student's record and the department identified from the department identification model is cross checked for the verification of the person. The verification conditions used and the respective messages shown are as follows:

if faceDepartment equals "Unknown" and uniformDepartment equals "Unknown"

message shown → Outsider Detected.

if faceDepartment equals "Unknown" and uniformDepartment is not "Unknown"

message shown → Potential Security Threat detected (someone posing to be a student).

if faceDepartment is not "Unknown" and uniformDepartment equals "Unknown"

message shown → {student_name} is not in university uniform.

if faceDepartment not equals uniformDepartment

reason shown → {student_name} appears to be wearing a uniform from a different department.

6.1 Output images of the Project

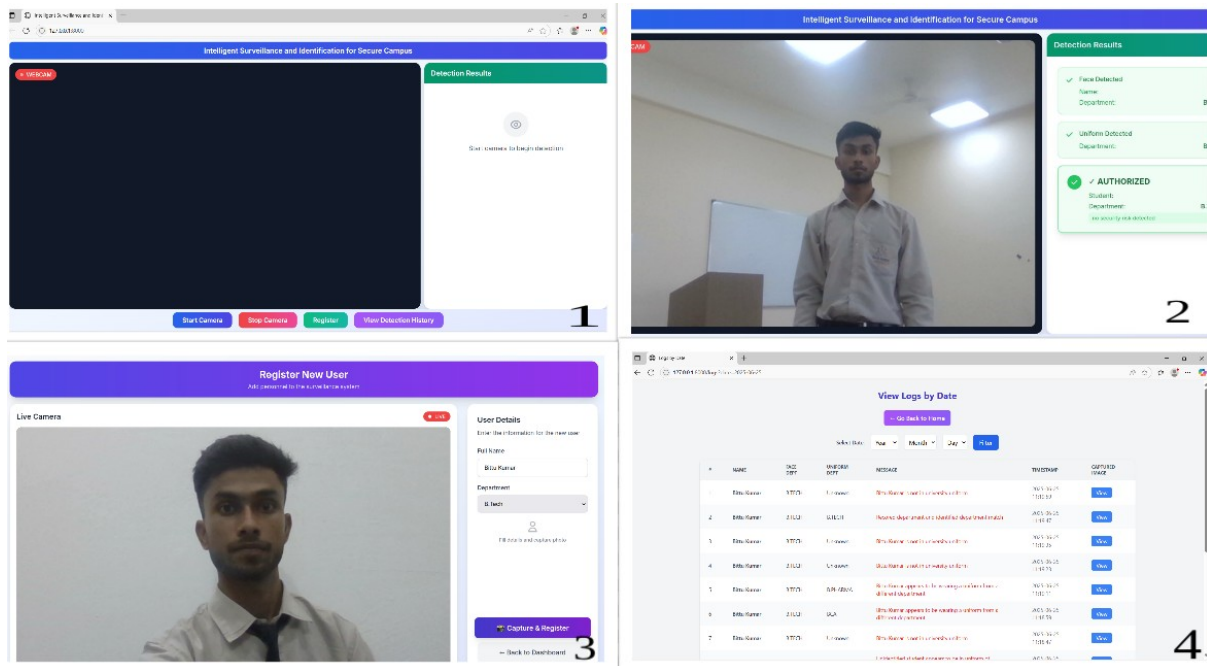


Fig-6.1: Output Images of Intelligent Surveillance and Identification System

The images collectively illustrate the user interface and workflow of an Intelligent Surveillance and Identification System designed for secure campus monitoring.

1. The first screen represents the main dashboard, where the system waits for camera activation before beginning face and uniform detection. The left section contains the live webcam panel, initially dark because the camera is not yet active. A small red “WEB CAM” indicator shows that the system is ready to stream once permission is granted. At the top, a clearly labeled banner “Intelligent Surveillance and Identification for Secure Campus” emphasizes the purpose of the system. On the right, the “Detection Results” panel guides the user with an icon and message instructing them to start the camera to begin real-time analysis. The lower portion contains action buttons such as Start Camera, Stop Camera, Register, and View Detection History, indicating that the system supports both monitoring and administrative functionalities.

2. When the camera is started, as shown in the second image, the interface becomes more dynamic. The left side displays a live video feed with the user's face visible. The right side

now shows that the system has successfully detected a face, displaying user details such as Name: Bittu and Department: B.Tech, which have been retrieved from the system's database. Below that, the interface shows an ongoing uniform detection process, visually represented by a loading card stating “Detecting Uniform Analyzing uniform patterns...”. This demonstrates the system’s dual-stage process: first confirming identity through facial recognition and then validating dress-code compliance through uniform classification.

3. The third image transitions to the user registration page, where new personnel can be added to the system. Here, the camera is active on the left, allowing the system to capture an image of the new registrant. On the right, there is a “User Details” form where fields such as Full Name and Department must be filled before capturing a registration photo. The “Capture & Register” button initiates the process, confirming that the system stores both facial features and associated academic or professional details.

4. The final image presents an updated version of the registration page, titled Register New Student, reflecting a more general onboarding workflow. It has the same live camera module, but the right panel is empty and waiting for user input. This consistent design across pages highlights usability, clear navigation, and seamless integration between detection and registration features.

6.2 Confusion Matrix

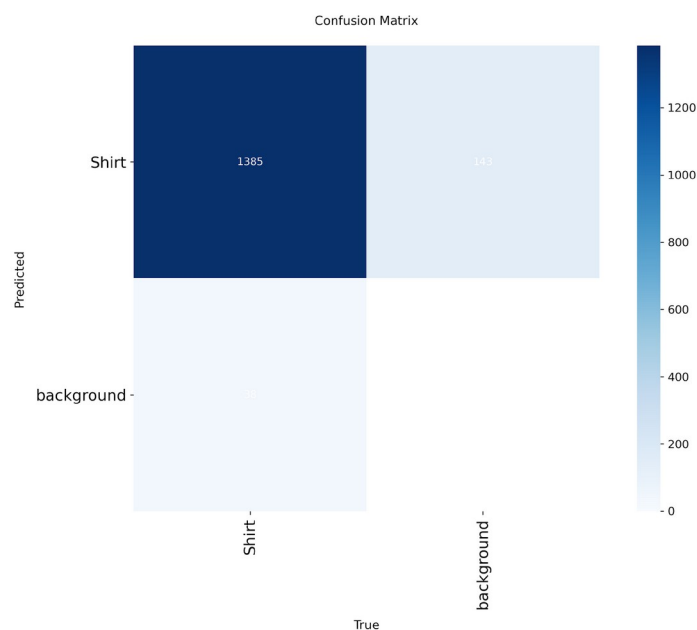


Fig-6.2: Confusion Matrix of Intelligent Surveillance and Identification for Secure Campus

This image shows a confusion matrix that evaluates how well a shirt-detection model performs when classifying images into two categories: “Shirt” and “background.” The horizontal axis represents the true labels from the dataset, while the vertical axis represents the model’s predicted labels. The top-left cell contains 1385, indicating the model correctly identified 1385 shirt images (true positives). The top-right cell shows 143, representing shirt images that the model incorrectly classified as background (false negatives). In the bottom-left cell, the value 38 shows background images incorrectly predicted as shirts (false positives). The bottom-right cell represents true background classifications, but it appears to be zero or nearly zero, which may indicate the background class is underrepresented or the value is too small to be visible in this visualization. The color intensity reflects the frequency of predictions, with darker shades indicating higher counts. Here, the shirt true-positive cell is the darkest, showing strong model performance for detecting shirts. The lighter shades in misclassification areas reveal where the model struggles. Overall, the confusion matrix suggests the model is highly accurate at identifying shirts but less consistent with distinguishing background regions, likely due to dataset imbalance or the visual similarity between shirts and background areas.

Chapter 7

Conclusion & Future Scopes

7.1 Conclusion

This project successfully developed an AI-driven surveillance and identification system aimed at enhancing campus security through automated student monitoring. By integrating computer vision techniques, deep learning models such as DeepFace, and real-time database querying, the system was capable of detecting students, classifying their uniforms by department, and verifying identity through facial recognition. The system analyzed continuous CCTV video streams, extracting uniform color data and comparing it with predefined department mappings. Simultaneously, facial features were captured and matched against stored profiles in the institutional database. Whenever discrepancies were detected between the student's actual department and the one inferred from their uniform, the system automatically flagged the individual and displayed violation alerts, thereby supporting quick administrative response. The implementation of an interactive, centralized monitoring dashboard greatly enhanced accessibility and operational efficiency for campus administrators. It provided real-time statistics, alert notifications, student logs, and system controls, contributing to a more structured and manageable surveillance workflow. Testing and evaluation further revealed that the system could operate reliably in real-time conditions, achieving high accuracy in both face detection and uniform classification tasks. Despite encountering issues such as inconsistent lighting, occlusions, camera placement limitations, and occasional false alerts, these challenges serve as valuable indicators for further improvements. Enhancing image quality preprocessing, expanding the training dataset, integrating multi-angle camera support, and refining color detection thresholds can significantly strengthen future system iterations. Overall, the project achieved its intended objectives and demonstrated the potential of artificial intelligence in transforming traditional campus security practices. It highlights how automated, data-driven surveillance can increase monitoring efficiency, reduce manual workload, and offer faster response to security breaches. This work establishes a strong foundation for scalable, intelligent surveillance systems and encourages further research into integrating behavioral analytics, multi-modal identification mechanisms, and advanced alerting frameworks to build a more secure and technologically empowered academic environment.

7.2 Future Scope

The proposed surveillance system offers substantial opportunities for expansion, enhancement, and integration, making it capable of evolving into a more robust, intelligent, and scalable security solution. One major future enhancement for the system is the introduction of an automated attendance mechanism. By extending the current facial recognition module, the system can automatically record student attendance the moment they are detected on camera. This eliminates the need for manual attendance marking, reduces human error, and prevents proxy attendance, making daily administrative tasks more efficient. Another important direction is expanding the solution into a campus-wide surveillance network. Instead of focusing only on gates or key checkpoints, the system can be deployed across classrooms, corridors, hostels, libraries, and common areas. This extended coverage would allow continuous monitoring of student movement and improve overall campus visibility for security teams. A third scope for improvement is the integration of real-time alert notifications. By sending automatic alerts through SMS, email, or a dedicated mobile application, administrators and security staff can be immediately informed when violations occur such as unauthorized entry, identity mismatches, or uniform discrepancies. This instant communication greatly enhances timely response. The system can also evolve to include behavioral analysis using AI, allowing it to detect suspicious activities, abnormal movement patterns, unauthorized group gatherings, or potential safety threats. This predictive capability would help identify risks before they escalate, transforming the system from a passive monitoring tool into an active security assistant. Integration with the institution's ERP (Enterprise Resource Planning) system represents another significant upgrade. This would allow attendance data, disciplinary records, and access logs to be updated automatically, improving record accuracy and reducing administrative burden. Finally, the system can be extended to support deeper uniform and dress-code enforcement. Beyond detecting uniform colors, future versions could identify missing ID cards, incomplete uniforms, or improper footwear, ensuring consistent adherence to institutional rules.

7.3 Limitations

While the proposed system demonstrates strong potential in automating surveillance and identity verification, several limitations need to be acknowledged to fully understand its operational boundaries.

1. The system's accuracy is significantly affected by lighting variability which can distort uniform colors and facial features.
2. Improper camera angles, long distances, obstructions, and motion blur reduce the system's ability to reliably detect faces and uniform colors.
3. The system is prone to false positives and false negatives due to similar uniform shades, low-quality CCTV footage, and misclassification of non-students.
4. The system poses privacy, ethical, and security risks because it processes sensitive facial and identity data that require strong protection and compliance measures.

References

1. J. J. K, K. N. S, B. Vadivel, G. G and P. V, "Efficient YOLOv8 Model for Licence, Helmet and Person Detection," 2025 5th International Conference on Pervasive Computing and Social Networking (ICPCSN), Salem, India, 2025, pp. 404-408, doi: 10.1109/ICPCSN65854.2025.11035282.
2. Z. Li and J. Wang, "Multi-person gesture recognition for complex environmental based on improved Yolov8 algorithm," 2025 5th International Academic Exchange Conference on Science and Technology Innovation (IAECST), Guangzhou, China, 2025, pp. 614-619, doi: 10.1109/IAECST60924.2025.10502852.
3. J. Nandre, S. Rai and B. R. Kanawade, "Comparative Analysis of Transfer Learning CNN for Face Recognition," 2024 2nd International Conference on Intelligent Technologies (CONIT), Hubli, India, 2024, pp. 1-6, doi: 10.1109/CONIT55038.2024.9847946.
4. D. S. Parker, "Integrating AI and DBMS through stream processing," [2024] Proceedings. Fifth International Conference on Data Engineering, Los Angeles, CA, USA, 2024, pp. 259-260, doi: 10.1109/ICDE.2024.47224.
5. T. Babu, E. P. M and R. R. Nair, "Real-Time Facial Expression Recognition Using Deep Learning for Enhanced Human-Machine Interactions," 2024 International BIT Conference (BITCON), Dhanbad, India, 2024, pp. 1-6, doi: 10.1109/BITCON63716.2024.10984634.
6. H. R. Tohidypour, M. T. Pourazad and P. Nasiopoulos, "A Security-Centric Deep Learning Enabled Camera Solution for Real-Time Human Fall Detection," 2023 18th International Conference on Wireless and Mobile Computing, Networking and Communications (WiMob), Thessaloniki, Greece, 2023, pp. 42-46, doi: 10.1109/WiMob55322.2023.9941359.
7. M. Doi, T. Matsumoto, A. Kimachi, S. Nishi and N. Ikoma, "Robust color object tracking method against illumination color Change," 2023 Joint 7th International Conference on Soft Computing and Intelligent Systems (SCIS) and 15th International Symposium on Advanced Intelligent Systems (ISIS), Kitakyushu, Japan, 2023, pp. 718-722, doi: 10.1109/SCIS-ISIS.2023.7044769.

- 8.** Y. Zhao, H. Liu, Q. Wang and C. Li, "A Multi-Object Tracking Method Using IoU and Color Fusion Model," 2023 9th International Conference on Signal and Image Processing (ICSIP), Nanjing, China, 2023, pp. 747-751, doi: 10.1109/ICSIP61881.2023.10671489.
- 9.** Y. Qi, Y. Wang and Y. Liu, "Object Tracking Based on Deep CNN Feature and Color Feature," 2022 14th IEEE International Conference on Signal Processing (ICSP), Beijing, China, 2022, pp. 469-473, doi: 10.1109/ICSP.2022.8652470.
- 10.** Zhiyuan Han, Tingrong Xu and Zhaohua Chen, "An improved color-based tracking by particle filter," Proceedings 2022 International Conference on Transportation, Mechanical, and Electrical Engineering (TMEE), Changchun, China, 2022, pp. 2512-2515, doi: 10.1109/TMEE.2022.6199732.
- 11.** S. Pati, A. Majumder, M. Gangopadhyay, G. Sen and A. Chakraborty, "Advanced Attendance Monitoring: A Unified System Using QR Codes and Facial Recognition," 2021 8th International Conference on Electronics, Materials Engineering & Nano-Technology (IEMENTech), Kolkata, India, 2021, pp. 1-4, doi: 10.1109/IEMENTech65115.2021.10959151.
- 12.** R. Nidhya, D. J. A. Pabi, U. DivyaSree, B. Abhishek, A. Harika and S. A. Alim, "Exploring Facial Recognition Technologies for Classroom Management," 2021 10th International Conference on Communication and Signal Processing (ICCSP), Melmaruvathur, India, 2021, pp. 1702-1707, doi: 10.1109/ICCSP60870.2021.10544143.

Appendix

Application Structure

IntelSurvIdSys

- |— app.py
- |— identify_department.py
- |— log.py
- |— match_face.py
- |— register_user.py
- |— templates
 - | |— index.html
 - | |— logs.html
 - | |— register.html
- |— uniform.py
- |— utils
 - | |— get_hsv.py
- |— pyproject.toml

Source Code

app.py

```
from ultralytics import YOLO
import numpy as np
import cv2

# custom trained YOLO model for shirt detection
model = YOLO("runs/content/runs/detect/train/weights/best.pt")

def identify_department_from_uniform_colorcode(image):
    try:
        # Run YOLO inference
        results = model(image)
        departments = []
```

```

# Process the results
for result in results:
    boxes = result.bboxes
    for idx, box in enumerate(boxes):
        x1, y1, x2, y2 = map(int, box.xyxy[0])
        class_id = int(box.cls)
        class_name = model.names[class_id].lower()
        confidence = float(box.conf[0])

        # Detect only shirts
        if class_name != "shirt":
            continue

        # Crop shirt region (BGR format)
        cropped_shirt = image[y1:y2, x1:x2]

        if cropped_shirt.size == 0:
            continue

        crop_area = (x2 - x1) * (y2 - y1)
        image_area = image.shape[0] * image.shape[1]

        # Area thresholds to filter out large/small false detections
        min_shirt_area = 0.01 * image_area # 1%
        max_shirt_area = 0.3 * image_area # 50%

        if crop_area < min_shirt_area or crop_area > max_shirt_area:
            continue # Skip invalid shirt sizes

        department = get_department(cropped_shirt, DEPARTMENT_COLORS)

        overall_confidence = confidence * (0.8 if department else 0.3)
        color_confidence = "high" if department else "low"

```

```
crop_area = (x2 - x1) * (y2 - y1)
```

```
result_dict = {  
    'department': department,  
    # 'dominant_color': [int(x) for x in dominant_color],  
    'shirt_detection_confidence': confidence,  
    'overall_confidence': overall_confidence,  
    'color_confidence': color_confidence,  
    'bounding_box': [x1, y1, x2, y2],  
    'crop_area': crop_area,  
}  
departments.append(result_dict)
```

```
if not departments:
```

```
    print("No shirts detected in the image.")
```

```
    return None
```

```
return departments[0]
```

```
except Exception as e:
```

```
    print(f"Error in identify_department_from_uniform_colorcode: {e}")
```

```
    return None
```

```
def find_max_contour(image, hsv_lower, hsv_upper, area_threshold=0.4):
```

```
    """
```

Detects contours in an image based on HSV range, finds the contour with the maximum area,

and checks if its area exceeds the specified threshold.

Parameters:

image (numpy.ndarray): Input image in BGR format.

hsv_lower (tuple): Lower bound of HSV values (H, S, V).

hsv_upper (tuple): Upper bound of HSV values (H, S, V).

area_threshold (float): Minimum area threshold for the contour.

Returns:

tuple: (max_contour, max_area, is_above_threshold)

- max_contour: Contour with the maximum area (None if no contours found).
- max_area: Area of the maximum contour (0 if no contours found).
- is_above_threshold: Boolean indicating if max_area exceeds area_threshold.

"""

Convert the image to HSV color space

hsv_image = cv2.cvtColor(image, cv2.COLOR_BGR2HSV)

Create a mask based on the HSV range

mask = cv2.inRange(hsv_image, np.array(hsv_lower), np.array(hsv_upper))

Find contours in the mask

contours, _ = cv2.findContours(mask, cv2.RETR_EXTERNAL,
cv2.CHAIN_APPROX_SIMPLE)

Initialize variables for maximum contour and area

max_contour = None

max_area = 0

Iterate through contours to find the one with maximum area

for contour in contours:

area = cv2.contourArea(contour)

if area > max_area:

max_area = area

max_contour = contour

Check if the maximum area exceeds the threshold

is_above_threshold = max_area > area_threshold

return max_contour, max_area, is_above_threshold

```

def get_department(image, department_colors):
    """
    Identify the department based on the dominant color of the shirt in the image.

    Parameters:
    - image: Input image in BGR format.
    - department_colors: Dictionary containing department color ranges.

    Returns:
    - str: Detected department name or "No Match".
    """
    # Convert the image to HSV color space
    # hsv_image = cv2.cvtColor(image, cv2.COLOR_BGR2HSV)

    department = {}

    # Find contours for each department color
    for dept, ranges in department_colors.items():
        lower = ranges['lower']
        upper = ranges['upper']

        max_contour, max_area, is_above_threshold = find_max_contour(image, lower, upper)

        if is_above_threshold:
            if not department or max_area > department['max_area']:
                department = {
                    'name': dept,
                    'max_contour': max_contour,
                    'max_area': max_area,
                    'hsv_lower': lower,
                    'hsv_upper': upper
                }

```

```

return department['name'] if department else None

# defined DEPARTMENT_COLORS with lower and upper HSV ranges for each department
DEPARTMENT_COLORS = {
    "Unknown": {'lower': [58, 20, 16], 'upper': [144, 210, 96]},
    "B.TECH": {'lower': [0, 0, 0], 'upper': [40, 255, 168]},
    "B.PHARMA": {'lower': [105, 48, 0], 'upper': [179, 255, 255]},
    "BCA": {'lower': [87, 19, 156], 'upper': [179, 255, 255]},
}

# Example usage:
from PIL import Image

if __name__ == "__main__":
    try:
        test_img = "tests/images/btech.jpeg"

        # Load the PIL image
        pil_image = Image.open(test_img)

        # Convert PIL Image to OpenCV BGR format
        img_array = np.array(pil_image)          # RGB format
        img_cv = cv2.cvtColor(img_array, cv2.COLOR_RGB2BGR)

        result = identify_department_from_uniform_colorcode(img_cv)

        if result and result.get('bounding_box'):
            print(f"\nShirt Detection Confidence: {result['shirt_detection_confidence']}")
            print(f"Bounding Box: {result['bounding_box']}")
            # print(f"Dominant Color: {result['dominant_color']}")
            print(f"Color Confidence: {result['color_confidence']}")
            print(f"Overall Confidence: {result['overall_confidence']}")
            print(f"Detected Department: {result['department']}")

```

```

# Convert PIL image to OpenCV format (RGB to BGR)
img_array = np.array(pil_image)
if img_array.shape[-1] == 3: # Ensure it's RGB
    img_cv = cv2.cvtColor(img_array, cv2.COLOR_RGB2BGR)
else:
    print("Image format not supported for display")
    exit(1)

# Extract bounding box coordinates (x1, y1, x2, y2)
x1, y1, x2, y2 = result['bounding_box']

# Draw bounding box on the image
start_point = (int(x1), int(y1))
end_point = (int(x2), int(y2))
color = (0, 255, 0) # Green color in BGR
thickness = 2
img_cv = cv2.rectangle(img_cv, start_point, end_point, color, thickness)

# Add department text above the bounding box
department_text = result['department']
font = cv2.FONT_HERSHEY_SIMPLEX
font_scale = 1.5 # Larger scale for big text
text_color = (0, 0, 255) # Black color in BGR
text_thickness = 3 # Increased thickness for bold text

# Calculate text position (slightly above the top-left corner of the bounding box)
text_position = (int(x1), int(y1) - 10) # Move 10 pixels above y1
img_cv = cv2.putText(img_cv, department_text, text_position, font, font_scale,
text_color, text_thickness, cv2.LINE_AA)

# Display the image with bounding box
cv2.imshow("Image with Bounding Box", img_cv)
cv2.waitKey(0) # Wait for key press to close

```

```

        cv2.destroyAllWindows()

    else:
        print("No department or bounding box detected")

except FileNotFoundError:
    print(f"Test image not found - replace '{test_img}' with your image path")
except Exception as e:
    print(f"An error occurred: {str(e)}")

import mysql.connector
import base64

# Read and encode the image in base64
# with open("faces/rohit.jpg", "rb") as image_file:
#     encoded_string = base64.b64encode(image_file.read()).decode("utf-8")

def register_user_base64encoded(name: str, image_data: str, department: str):
    try:
        # Strip data URL header if present
        _header, encoded = (
            image_data.split(",", 1) if "," in image_data else ("", image_data)
        )

        # Connect to the database
        conn = mysql.connector.connect(
            host="localhost", user="bittu", password="bittu", database="deepface_schema"
        )

        cursor = conn.cursor()

        # Insert the name and base64-encoded image string
        cursor.execute(

```

```

        "INSERT INTO registered_users (name, face, department) VALUES (%s, %s, %s)",
        (name, encoded, department),
    )

    conn.commit()
    conn.close()
    return {
        "success": True,
        "message": f"User {name} registered successfully in {department} department.",
    }
except mysql.connector.Error as err:
    print(f"Error: {err}")
    return {"success": False, "message": f"Failed to register user {name}: {err}"}

```

log.py

```

import mysql.connector
from mysql.connector import Error

def connect_db():
    try:
        conn = mysql.connector.connect(
            host="localhost", user="bittu", password="bittu", database="deepface_schema"
        )
        return conn
    except Error as e:
        raise Exception(f"Database connection failed: {e}")

def save_log_to_database(data):
    try:
        conn = connect_db()
        cursor = conn.cursor()

```

```

# Ensure the `logs` table exists or change the table name
sql = """
    INSERT INTO logs_data (name, face_dept, uniform_dept, status, captured_uuid)
    VALUES (%s, %s, %s, %s, %s)
    """

values = (
    data["name"],
    data["faceDept"],
    data["uniformDept"],
    data["status"],
    data["captured_uuid"],
)

cursor.execute(sql, values)
conn.commit()

except Error as e:
    raise Exception(f"Failed to insert log: {e}")
finally:
    if cursor:
        cursor.close()
    if conn:
        conn.close()

```

index.html

```

<!DOCTYPE html>
<html lang="en">

<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Intelligent Surveillance and Identification for Secure Campus</title>
    <script src="https://cdn.tailwindcss.com"></script>

```

```

<link href="https://fonts.googleapis.com/css2?
family=Inter:wght@300;400;500;600;700;800&display=swap"
rel="stylesheet">
<script>
  tailwind.config = {
    theme: {
      extend: {
        fontFamily: {
          'inter': ['Inter', 'sans-serif'],
        },
        animation: {
          'fade-in': 'fadeIn 0.5s ease-in-out',
          'slide-up': 'slideUp 0.3s ease-out',
          'pulse-slow': 'pulse 2s cubic-bezier(0.4, 0, 0.6, 1) infinite',
        },
        keyframes: {
          fadeIn: {
            '0%': { opacity: '0' },
            '100%': { opacity: '1' },
          },
          slideUp: {
            '0%': { transform: 'translateY(10px)', opacity: '0' },
            '100%': { transform: 'translateY(0)', opacity: '1' },
          }
        }
      }
    }
  }
</script>
</head>

<body class="h-screen bg-gradient-to-br from-slate-50 via-blue-50 to-indigo-100 font-inter">
  <div class="w-full h-full flex flex-col p-2">
    <!-- Top Panel: Title -->

```



```

<div
  class="bg-gradient-to-r from-blue-600 to-indigo-600 px-4 py-2 rounded-xl shadow
text-white font-semibold text-center text-lg mb-2">
  Intelligent Surveillance and Identification for Secure Campus
</div>

<!-- Main Grid: Results on Left, Video on Right -->
<div class="flex flex-1 gap-2 overflow-hidden">

  <!-- Video Feed - Right Panel -->
  <div class="w-9/12 bg-white rounded-2xl shadow-xl border border-gray-200 flex
flex-col">
    <div
      class="p-4 flex-1 flex items-center justify-center relative bg-gray-900
rounded-2xl overflow-hidden">
      <video id="video" class="w-full h-full object-cover rounded-xl"
autoplay></video>
      <div
        class="absolute top-4 left-4 bg-red-500 text-white px-3 py-1 rounded-full text-
sm font-medium flex items-center">
          <div class="w-2 h-2 bg-white rounded-full mr-2 animate-pulse"></div>
          WEBCAM
        </div>
      </div>
    </div>

  <!-- Detection Results - Wider Left Panel -->
  <div class="w-4/12 bg-white rounded-2xl shadow-xl border border-gray-200 flex
flex-col">
    <div class="bg-gradient-to-r from-emerald-600 to-teal-600 px-4 py-3 rounded-
t-2xl">
      <h2 class="text-lg font-semibold text-white">Detection Results</h2>
    </div>
    <div id="result" class="p-6 overflow-auto text-sm flex-1 min-h-[0px] space-y-4">
      <div class="text-center text-gray-500 py-12">

```

```

        <div class="w-16 h-16 bg-gray-100 rounded-full flex items-center justify-
center mx-auto mb-4">
            <svg class="w-8 h-8 text-gray-400" fill="none" stroke="currentColor"
viewBox="0 0 24 24">
                <path stroke-linecap="round" stroke-linejoin="round" stroke-width="2"
                d="M15 12a3 3 0 11-6 0 3 3 0 016 0z"></path>
                <path stroke-linecap="round" stroke-linejoin="round" stroke-width="2"
                d="M2.458 12C3.732 7.943 7.523 5 12 5c4.478 0 8.268 2.943 9.542 7-
1.274 4.057-5.064 7-9.542 7-4.477 0-8.268-2.943-9.542-7z">
                </path>
            </svg>
        </div>
        <p class="text-base text-gray-600">Start camera to begin detection</p>
    </div>
</div>
</div>
</div>
</div>

<!-- Control Buttons -->
<div class="mt-2 flex justify-center items-center gap-4">
    <button id="startButton"
        class="bg-gradient-to-r from-blue-600 to-indigo-600 hover:from-blue-700
hover:to-indigo-700 text-white px-6 py-2 rounded-xl font-semibold">
        Start Camera
    </button>
    <button id="stopButton"
        class="bg-gradient-to-r from-red-500 to-pink-500 hover:from-red-600 hover:to-
pink-600 text-white px-6 py-2 rounded-xl font-semibold">
        Stop Camera
    </button>
    <button id="registerButton"
        class="bg-gradient-to-r from-emerald-500 to-teal-500 hover:from-emerald-600
hover:to-teal-600 text-white px-6 py-2 rounded-xl font-semibold">
        Register
    </button>

```

```

        <button id="viewLogsBtn"
        class="bg-gradient-to-r from-purple-500 to-violet-500 hover:from-purple-600
        hover:to-violet-600 text-white px-6 py-2 rounded-xl font-semibold">
            View Detection History
        </button>
    </div>
</div>

<script>
    const video = document.getElementById('video');
    const startButton = document.getElementById('startButton');
    const stopButton = document.getElementById('stopButton');
    const registerButton = document.getElementById('registerButton');
    const resultDiv = document.getElementById('result');
    let mediaStream = null;
    let intervalId = null;
    let isProcessing = false;

    startButton.addEventListener('click', async () => {
        if (navigator.mediaDevices && navigator.mediaDevices.getUserMedia) {
            try {
                mediaStream = await navigator.mediaDevices.getUserMedia({ video: true });
                video.srcObject = mediaStream;
                intervalId = setInterval(() => {
                    if (!isProcessing) {
                        detect_and_display();
                    }
                }, 3000);
            } catch (error) {
                console.error('Error accessing camera:', error);
                resultDiv.innerHTML = `
                    <div class="bg-red-50 border border-red-200 text-red-700 p-4 rounded-xl text-
                    sm shadow-sm">
                        <div class="flex items-center">

```

```

        <svg class="w-5 h-5 text-red-500 mr-3 flex-shrink-0" fill="currentColor"
viewBox="0 0 20 20">
            <path fill-rule="evenodd" d="M10 18a8 8 0 100-16 8 8 0 00
16zM8.707 7.293a1 1 0 00-1.414 1.414L8.586 10l-1.293 1.293a1 1 0 101.414 1.414L10
11.414l1.293 1.293a1 1 0 001.414-1.414L11.414 10l1.293-1.293a1 1 0 00-1.414-1.414L10
8.586 8.707 7.293z" clip-rule="evenodd"></path>
        </svg>
        <div>
            <p class="font-medium">Camera Error</p>
            <p class="text-xs mt-1 text-red-600">${error.message}</p>
        </div>
    </div>
</div>
`
    }
}
});

```

```

stopButton.addEventListener('click', () => {
    if (mediaStream) {
        mediaStream.getTracks().forEach(track => track.stop());
        video.srcObject = null;
    }
    if (intervalId) {
        clearInterval(intervalId);
    }
    isProcessing = false;
    resultDiv.innerHTML = `
        <div class="text-center text-gray-500 py-12">
            <div class="w-16 h-16 bg-gray-100 rounded-full flex items-center justify-center
mx-auto mb-4">
                <svg class="w-8 h-8 text-gray-400" fill="none" stroke="currentColor"
viewBox="0 0 24 24">
                    <path stroke-linecap="round" stroke-linejoin="round" stroke-width="2"
d="M18.364 18.364A9 9 0 005.636 5.636m12.728 12.728L5.636 5.636m12.728
12.728L18.364 5.636M5.636 18.364l12.728-12.728"></path>

```

```

        </svg>
    </div>
    <p class="text-base text-gray-600">Camera stopped</p>
</div>
`
});

registerButton.addEventListener('click', () => {
    window.location.href = '/register';
});

document.getElementById("viewLogsBtn").addEventListener("click", function () {
    window.location.href = "/logs";
});

async function detect_and_display() {
    if (isProcessing) return;
    isProcessing = true;

    const canvas = document.createElement('canvas');
    canvas.width = video.videoWidth;
    canvas.height = video.videoHeight;
    const context = canvas.getContext('2d');
    context.drawImage(video, 0, 0, canvas.width, canvas.height);
    const imageData = canvas.toDataURL('image/jpeg');

    try {
        resultDiv.innerHTML = `
            <div class="space-y-4">
                <div class="bg-blue-50 border border-blue-200 text-blue-700 p-4 rounded-xl shadow-sm animate-pulse">
                    <div class="flex items-center">
                        <div class="animate-spin h-5 w-5 border-2 border-blue-500 border-t-transparent rounded-full mr-3 flex-shrink-0"></div>

```

```

        <div>
          <p class="font-medium">Detecting Face</p>
          <p class="text-xs mt-1 opacity-75">Analyzing facial features...</p>
        </div>
      </div>
    </div>
  </div>
`
;

const faceRes = await fetch('/match', {
  method: 'POST',
  headers: { 'Content-Type': 'application/json' },
  body: JSON.stringify({ image: imageData })
}).then(res => res.json());

const name = faceRes.name || "Unknown";
const faceDept = faceRes.department || "Unknown";
const captured_uuid = faceRes.captured_uuid || "default";

resultDiv.innerHTML = `
  <div class="space-y-4">
    <div class="bg-green-50 border border-green-200 text-green-700 p-4 rounded-
xl shadow-sm">
      <div class="flex items-start">
        <svg class="w-5 h-5 text-green-500 mr-3 flex-shrink-0 mt-0.5"
fill="currentColor" viewBox="0 0 20 20">
          <path fill-rule="evenodd" d="M16.707 5.293a1 1 0 010 1.414l-8 8a1 1
0 01-1.414 0l-4-4a1 1 0 011.414-1.414L8 12.586l7.293-7.293a1 1 0 011.414 0z" clip-
rule="evenodd"></path>
        </svg>
        <div class="flex-1">
          <p class="font-medium mb-2">Face Detected</p>
          <div class="text-sm space-y-1">
            <div class="flex justify-between">
              <span class="text-green-600">Name:</span>

```

```

        <span class="font-medium">${name}</span>
      </div>
      <div class="flex justify-between">
        <span class="text-green-600">Department:</span>
        <span class="font-medium">${faceDept}</span>
      </div>
    </div>
  </div>
</div>
</div>
</div>

<div class="bg-blue-50 border border-blue-200 text-blue-700 p-4 rounded-xl
shadow-sm animate-pulse">
  <div class="flex items-center">
    <div class="animate-spin h-5 w-5 border-2 border-blue-500 border-t-
transparent rounded-full mr-3 flex-shrink-0"></div>
    <div>
      <p class="font-medium">Detecting Uniform</p>
      <p class="text-xs mt-1 opacity-75">Analyzing uniform patterns...</p>
    </div>
  </div>
</div>
</div>
</div>
`;

```

```

await new Promise(resolve => setTimeout(resolve, 800));

```

```

const uniformRes = await fetch('/detect_department', {
  method: 'POST',
  headers: { 'Content-Type': 'application/json' },
  body: JSON.stringify({ image: imageData })
}).then(res => res.json());

```

```

const uniformDept = uniformRes.department || "Unknown";

```

```

resultDiv.innerHTML = `
  <div class="space-y-4">
    <div class="bg-green-50 border border-green-200 text-green-700 p-4 rounded-
xl shadow-sm">
      <div class="flex items-start">
        <svg class="w-5 h-5 text-green-500 mr-3 flex-shrink-0 mt-0.5"
fill="currentColor" viewBox="0 0 20 20">
          <path fill-rule="evenodd" d="M16.707 5.293a1 1 0 010 1.414l-8 8a1 1
0 01-1.414 0l-4-4a1 1 0 011.414-1.414L8 12.586l7.293-7.293a1 1 0 011.414 0z" clip-
rule="evenodd"></path>
        </svg>
        <div class="flex-1">
          <p class="font-medium mb-2">Face Detected</p>
          <div class="text-sm space-y-1">
            <div class="flex justify-between">
              <span class="text-green-600">Name:</span>
              <span class="font-medium">${name}</span>
            </div>
            <div class="flex justify-between">
              <span class="text-green-600">Department:</span>
              <span class="font-medium">${faceDept}</span>
            </div>
          </div>
        </div>
      </div>
    </div>
  </div>
  <div class="bg-green-50 border border-green-200 text-green-700 p-4 rounded-
xl shadow-sm">
    <div class="flex items-start">
      <svg class="w-5 h-5 text-green-500 mr-3 flex-shrink-0 mt-0.5"
fill="currentColor" viewBox="0 0 20 20">
        <path fill-rule="evenodd" d="M16.707 5.293a1 1 0 010 1.414l-8 8a1 1
0 01-1.414 0l-4-4a1 1 0 011.414-1.414L8 12.586l7.293-7.293a1 1 0 011.414 0z" clip-
rule="evenodd"></path>

```



```

</svg>
<div class="flex-1">
  <p class="font-medium mb-2">Uniform Detected</p>
  <div class="text-sm">
    <div class="flex justify-between">
      <span class="text-green-600">Department:</span>
      <span class="font-medium">${uniformDept}</span>
    </div>
  </div>
</div>
</div>
</div>

<div class="bg-blue-50 border border-blue-200 text-blue-700 p-4 rounded-xl
shadow-sm animate-pulse">
  <div class="flex items-center">
    <div class="animate-spin h-5 w-5 border-2 border-blue-500 border-t-
transparent rounded-full mr-3 flex-shrink-0"></div>
    <div>
      <p class="font-medium">Verifying Authorization</p>
      <p class="text-xs mt-1 opacity-75">Cross-checking credentials...</p>
    </div>
  </div>
</div>
</div>
`;

```

```

await new Promise(resolve => setTimeout(resolve, 800));
let REASON = "";

let authorizationHtml = "";
if (faceDept === uniformDept && name !== "Unknown" && faceDept !==
"Unknown") {
  authorizationHtml = `

```

```

        <div class="bg-green-50 border-2 border-green-300 text-green-800 p-5
rounded-xl shadow-lg">
            <div class="flex items-start">
                <div class="w-8 h-8 bg-green-500 rounded-full flex items-center justify-
center mr-4 flex-shrink-0">
                    <svg class="w-5 h-5 text-white" fill="currentColor" viewBox="0 0 20
20">
                        <path fill-rule="evenodd" d="M16.707 5.293a1 1 0 010 1.414l-8
8a1 1 0 01-1.414 0l-4-4a1 1 0 011.414-1.414L8 12.586l7.293-7.293a1 1 0 011.414 0z" clip-
rule="evenodd"></path>
                    </svg>
                </div>
                <div class="flex-1">
                    <div class="text-sm space-y-1">
                        <div class="flex justify-between">
                            <span class="text-green-700">Student:</span>
                            <span class="font-semibold">${name}</span>
                        </div>
                        <div class="flex justify-between">
                            <span class="text-green-700">Department:</span>
                            <span class="font-semibold">${faceDept}</span>
                        </div>
                    </div>
                    <div class="mt-2 text-xs text-green-600 bg-green-100 px-2 py-1
rounded-md">
                        No security risk detected
                    </div>
                </div>
            </div>
        </div>
    `;
} else {
    let reason = 'Recorded department and identified department match';

    if (faceDept === "Unknown" && uniformDept === "Unknown") {

```

```

        reason = `Outsider detected`;
    } else if (faceDept === "Unknown" && uniformDept !== "Unknown") {
        reason = `Unidentified student appears to be in uniform of department $
{uniformDept}`;
    } else if (faceDept !== "Unknown" && uniformDept === "Unknown") {
        reason = `${name} is not in university uniform`;
    } else if (faceDept !== uniformDept) {
        reason = `${name} appears to be wearing a uniform from a different
department`;
    }

    REASON = reason;

    authorizationHtml = `
        <div class="bg-red-50 border-2 border-red-300 text-red-800 p-5 rounded-xl
shadow-lg">
            <div class="flex items-start">
                <div class="w-8 h-8 bg-red-500 rounded-full flex items-center justify-
center mr-4 flex-shrink-0">
                    <svg class="w-5 h-5 text-white" fill="currentColor" viewBox="0 0 20
20">
                        <path fill-rule="evenodd" d="M4.293 4.293a1 1 0 011.414 0L10
8.586l4.293-4.293a1 1 0 111.414 1.414L11.414 10l4.293 4.293a1 1 0 01-1.414 1.414L10
11.414l-4.293 4.293a1 1 0 01-1.414-1.414L8.586 10 4.293 5.707a1 1 0 010-1.414z" clip-
rule="evenodd"></path>
                    </svg>
                </div>
                <div class="flex-1">
                    <div class="text-sm">
                        <div class="text-red-700 bg-red-100 px-2 py-1 rounded-md">
                            ${reason}
                        </div>
                    </div>
                </div>
            </div>
        </div>
    `;

```

```
`;  
}
```

```
resultDiv.innerHTML = `  
  <div class="space-y-4">  
    <div class="bg-green-50 border border-green-200 text-green-700 p-4 rounded-  
xl shadow-sm">  
      <div class="flex items-start">  
        <svg class="w-5 h-5 text-green-500 mr-3 flex-shrink-0 mt-0.5"  
fill="currentColor" viewBox="0 0 20 20">  
          <path fill-rule="evenodd" d="M16.707 5.293a1 1 0 010 1.414l-8 8a1 1  
0 01-1.414 0l-4-4a1 1 0 011.414-1.414L8 12.586l7.293-7.293a1 1 0 011.414 0z" clip-  
rule="evenodd"></path>  
        </svg>  
        <div class="flex-1">  
          <p class="font-medium mb-2">Face Detected</p>  
          <div class="text-sm space-y-1">  
            <div class="flex justify-between">  
              <span class="text-green-600">Name:</span>  
              <span class="font-medium">${name}</span>  
            </div>  
            <div class="flex justify-between">  
              <span class="text-green-600">Department:</span>  
              <span class="font-medium">${faceDept}</span>  
            </div>  
          </div>  
        </div>  
      </div>  
    </div>  
  <div class="bg-green-50 border border-green-200 text-green-700 p-4 rounded-  
xl shadow-sm">  
    <div class="flex items-start">  
      <svg class="w-5 h-5 text-green-500 mr-3 flex-shrink-0 mt-0.5"  
fill="currentColor" viewBox="0 0 20 20">
```

```

    <path fill-rule="evenodd" d="M16.707 5.293a1 1 0 010 1.414l-8 8a1 1
0 01-1.414 0l-4-4a1 1 0 011.414-1.414L8 12.586l7.293-7.293a1 1 0 011.414 0z" clip-
rule="evenodd"></path>

```

```

</svg>

```

```

<div class="flex-1">

```

```

    <p class="font-medium mb-2">Uniform Detected</p>

```

```

    <div class="text-sm">

```

```

        <div class="flex justify-between">

```

```

            <span class="text-green-600">Department:</span>

```

```

            <span class="font-medium">${uniformDept}</span>

```

```

        </div>

```

```

    </div>

```

```

</div>

```

```

</div>

```

```

</div>

```

```

    ${authorizationHtml}

```

```

</div>

```

```

`;

```

```

await new Promise(resolve => setTimeout(resolve, 7000));

```

```

await storeLog(name, faceDept, uniformDept, REASON, captured_uuid);

```

```

} catch (error) {

```

```

    console.error('Error:', error);

```

```

    resultDiv.innerHTML = `

```

```

        <div class="bg-red-50 border border-red-200 text-red-700 p-4 rounded-xl
shadow-sm">

```

```

            <div class="flex items-center">

```

```

                <svg class="w-5 h-5 text-red-500 mr-3 flex-shrink-0" fill="currentColor"
viewBox="0 0 20 20">

```

```

                    <path fill-rule="evenodd" d="M10 18a8 8 0 100-16 8 8 0 00 16zM8.707
7.293a1 1 0 00-1.414 1.414L8.586 10l-1.293 1.293a1 1 0 101.414 1.414L10 11.414l1.293
1.293a1 1 0 001.414-1.414L11.414 10l1.293-1.293a1 1 0 00-1.414-1.414L10 8.586 8.707
7.293z" clip-rule="evenodd"></path>

```

```

                </svg>

```

```

        <div>
            <p class="font-medium">Processing Error</p>
            <p class="text-xs mt-1 text-red-600">${error.message}</p>
        </div>
    </div>
</div>
`;
    await new Promise(resolve => setTimeout(resolve, 4000));
  } finally {
    isProcessing = false;
  }
}

async function storeLog(name, faceDept, uniformDept, reason, captured_uuid) {
  try {
    if(reason === "") {
      reason = 'Recorded department and identified department match';
    };

    const response = await fetch('/store_log', {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify({
        name: name,
        face_dept: faceDept,
        uniform_dept: uniformDept,
        reason: reason,
        captured_uuid: captured_uuid,
      })
    });

    if (!response.ok) {
      // Handle HTTP errors
      const errorText = await response.text();
    }
  }
}

```

```

        throw new Error(`Request failed with status ${response.status}: ${errorText}`);
    }

    const data = await response.json();
    return data;

} catch (error) {
    console.error('Error storing log:', error);
    // Optionally re-throw or handle the error in your UI
    throw error;
}
}
</script>
</body>

</html>

```

register.html

```

<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0"/>
    <title>Register New User</title>
    <script src="https://cdn.tailwindcss.com"></script>
    <link href="https://fonts.googleapis.com/css2?family=Inter:wght@300;400;500;600;700;800&display=swap" rel="stylesheet"/>
    <script>
        tailwind.config = {
            theme: {
                extend: {
                    fontFamily: {
                        inter: ['Inter', 'sans-serif'],
                    },
                },
            },
        };
    </script>

```

```

        animation: {
          'fade-in': 'fadeIn 0.5s ease-in-out',
          'slide-up': 'slideUp 0.3s ease-out',
        },
        keyframes: {
          fadeIn: {
            '0%': { opacity: '0' },
            '100%': { opacity: '1' },
          },
          slideUp: {
            '0%': { transform: 'translateY(10px)', opacity: '0' },
            '100%': { transform: 'translateY(0)', opacity: '1' },
          }
        }
      }
    }
  }
</script>
</head>
<body class="h-screen bg-gradient-to-br from-slate-50 via-blue-50 to-indigo-100 font-inter overflow-hidden">
  <div class="h-full flex flex-col p-4">
    <!-- Header -->
    <div class="bg-gradient-to-r from-indigo-600 to-purple-600 px-6 py-4 rounded-2xl shadow-xl text-white text-center mb-4">
      <h1 class="text-2xl font-bold">Register New Student</h1>
      <p class="text-indigo-100 text-sm mt-1">Add Student to the System's Records</p>
    </div>

    <!-- Main Content -->
    <div class="flex-1 flex gap-6 min-h-0">
      <!-- Camera Feed -->
      <div class="flex-1 bg-white rounded-2xl shadow-xl border border-gray-200 p-4 flex flex-col">
        <div class="flex items-center justify-between mb-3">

```



```

    <h2 class="text-lg font-semibold text-gray-800">Live Camera</h2>
    <div class="bg-red-500 text-white px-3 py-1 rounded-full text-xs font-medium
flex items-center">
      <div class="w-2 h-2 bg-white rounded-full mr-2 animate-pulse"></div>
      LIVE
    </div>
  </div>
  <div class="flex-1 bg-gray-900 rounded-xl overflow-hidden relative">
    <video id="video" class="w-full h-full object-cover" autoplay></video>
    <div class="absolute inset-0 flex items-center justify-center"
id="cameraPlaceholder">
      <div class="text-white text-center">
        <svg class="w-16 h-16 mx-auto mb-4 opacity-50" fill="none"
stroke="currentColor" viewBox="0 0 24 24">
          <path stroke-linecap="round" stroke-linejoin="round" stroke-width="2"
d="M15 10l4.553-2.276A1 1 0 0121 8.618v6.764a1 1 0 01-1.447.894L15 14M5 18h8a2 2 0
002-2V8a2 2 0 00-2-2H5a2 2 0 00-2 2v8a2 2 0 002 2z"></path>
        </svg>
        <p class="text-lg font-medium opacity-75">Starting camera...</p>
      </div>
    </div>
  </div>
</div>

<!-- Registration Form -->
<div class="w-80 bg-white rounded-2xl shadow-xl border border-gray-200 p-6 flex
flex-col">
  <div class="mb-6">
    <h2 class="text-lg font-semibold text-gray-800 mb-2">Student Details</h2>
    <p class="text-sm text-gray-600">Enter the information for the new student</p>
  </div>

  <form id="registerForm" class="flex-1 flex flex-col">
    <div class="space-y-4 flex-1">
      <div>

```

```

Name</label>
    <label class="block text-sm font-medium text-gray-700 mb-2">Full
    <input
      type="text"
      id="name"
      required
      placeholder="Enter full name"
      class="w-full border border-gray-300 rounded-lg px-4 py-3 text-sm
focus:ring-2 focus:ring-indigo-500 focus:border-indigo-500 transition-colors"
    />
  </div>

  <div>
    <label class="block text-sm font-medium text-gray-700
mb-2">Department</label>
    <select
      id="department"
      required
      class="w-full border border-gray-300 rounded-lg px-4 py-3 text-sm
focus:ring-2 focus:ring-indigo-500 focus:border-indigo-500 transition-colors"
    >
      <option value="">Select department</option>
      <option value="B.TECH">B.Tech</option>
      <option value="B.PHARMA">B.Pharma</option>
      <option value="BCA">BCA</option>
    </select>
  </div>

  <div id="status" class="min-h-[60px] flex items-center justify-center text-sm
text-center">
    <div class="text-gray-500">
      <svg class="w-8 h-8 mx-auto mb-2 opacity-50" fill="none"
stroke="currentColor" viewBox="0 0 24 24">
        <path stroke-linecap="round" stroke-linejoin="round" stroke-
width="2" d="M16 7a4 4 0 11-8 0 4 4 0 018 0zM12 14a7 7 0 00-7 7h14a7 7 0
00-7-7z"></path>

```

```

        </svg>
        <p class="text-xs">Fill details and capture photo</p>
    </div>
</div>
</div>

<div class="space-y-3 pt-4 border-t border-gray-100">
    <button
        type="button"
        id="captureBtn"
        class="w-full bg-gradient-to-r from-indigo-600 to-purple-600 hover:from-indigo-700 hover:to-purple-700 text-white px-6 py-3 rounded-lg font-semibold transition-all transform hover:scale-105 shadow-lg"
    >
         Capture & Register
    </button>

    <a
        href="/"
        class="block w-full text-center bg-gray-100 hover:bg-gray-200 text-gray-700 px-6 py-3 rounded-lg font-medium transition-colors"
    >
        ← Back to Dashboard
    </a>
</div>
</form>
</div>
</div>

<script>
    const video = document.getElementById('video');
    const captureBtn = document.getElementById('captureBtn');
    const statusDiv = document.getElementById('status');
    const cameraPlaceholder = document.getElementById('cameraPlaceholder');

```

```

// Start camera
navigator.mediaDevices.getUserMedia({ video: true })
  .then(stream => {
    video.srcObject = stream;
    video.onloadedmetadata = () => {
      cameraPlaceholder.style.display = 'none';
    };
  })
  .catch(err => {
    console.error('Camera error:', err);
    cameraPlaceholder.innerHTML = `
      <div class="text-white text-center">
        <svg class="w-16 h-16 mx-auto mb-4 text-red-400" fill="none"
stroke="currentColor" viewBox="0 0 24 24">
          <path stroke-linecap="round" stroke-linejoin="round" stroke-width="2"
d="M12 9v2m0 4h.01m-6.938 4h13.856c1.54 0 2.502-1.667 1.732-2.5L13.732 4c-.77-.833-
1.964-.833-2.732 0L4.082 16.5c-.77.833.192 2.5 1.732 2.5z"></path>
        </svg>
        <p class="text-lg font-medium">Camera Error</p>
        <p class="text-sm opacity-75 mt-1">Unable to access camera</p>
      </div>
    `;
    statusDiv.innerHTML = `
      <div class="text-red-600 text-center">
        <svg class="w-8 h-8 mx-auto mb-2" fill="currentColor" viewBox="0 0 20
20">
          <path fill-rule="evenodd" d="M10 18a8 8 0 100-16 8 8 0 00 16zM8.707
7.293a1 1 0 00-1.414 1.414L8.586 10l-1.293 1.293a1 1 0 101.414 1.414L10 11.414l1.293
1.293a1 1 0 001.414-1.414L11.414 10l1.293-1.293a1 1 0 10-1.414-1.414L10 8.586 8.707
7.293z" clip-rule="evenodd"></path>
        </svg>
        <p class="text-xs font-medium">Camera access failed</p>
      </div>
    `;
  });

```

```

captureBtn.addEventListener('click', () => {
  const name = document.getElementById('name').value.trim();
  const dept = document.getElementById('department').value.trim();

  if (!name || !dept) {
    statusDiv.innerHTML = `
      <div class="text-amber-600 text-center">
        <svg class="w-8 h-8 mx-auto mb-2" fill="currentColor" viewBox="0 0 20
20">
          <path fill-rule="evenodd" d="M8.257 3.099c.765-1.36 2.722-1.36 3.486
0l5.58 9.92c.75 1.334-.213 2.98-1.742 2.98H4.42c-1.53 0-2.493-1.646-1.743-2.98l5.58-
9.92zM11 13a1 1 0 11-2 0 1 1 0 012 0zm-1-8a1 1 0 00-1 1v3a1 1 0 002 0V6a1 1 0 00-1-1z"
clip-rule="evenodd"></path>
        </svg>
        <p class="text-xs font-medium">Please fill all fields</p>
      </div>
    `;
    return;
  }

  // Show processing state
  statusDiv.innerHTML = `
    <div class="text-blue-600 text-center">
      <div class="animate-spin w-8 h-8 border-2 border-blue-500 border-t-transparent
rounded-full mx-auto mb-2"></div>
      <p class="text-xs font-medium">Registering user...</p>
    </div>
  `;

  const canvas = document.createElement('canvas');
  canvas.width = video.videoWidth;
  canvas.height = video.videoHeight;
  const ctx = canvas.getContext('2d');
  ctx.drawImage(video, 0, 0, canvas.width, canvas.height);

```

```

const imageData = canvas.toDataURL('image/jpeg');

// Post data to server
fetch('/register_user', {
  method: 'POST',
  headers: { 'Content-Type': 'application/json' },
  body: JSON.stringify({ image: imageData, name: name, department: dept })
})
.then(res => res.json())
.then(data => {
  if (data.success) {
    statusDiv.innerHTML = `
      <div class="text-green-600 text-center">
        <svg class="w-8 h-8 mx-auto mb-2" fill="currentColor" viewBox="0 0 20
20">
          <path fill-rule="evenodd" d="M16.707 5.293a1 1 0 010 1.414l-8 8a1 1 0
01-1.414 0l-4-4a1 1 0 011.414-1.414L8 12.586l7.293-7.293a1 1 0 011.414 0z" clip-
rule="evenodd"></path>
        </svg>
        <p class="text-xs font-medium">Successfully registered!</p>
      </div>
    `;
    // Clear form
    document.getElementById('name').value = "";
    document.getElementById('department').value = "";
  } else {
    statusDiv.innerHTML = `
      <div class="text-red-600 text-center">
        <svg class="w-8 h-8 mx-auto mb-2" fill="currentColor" viewBox="0 0 20
20">
          <path fill-rule="evenodd" d="M10 18a8 8 0 100-16 8 8 0 00 16zM8.707
7.293a1 1 0 00-1.414 1.414L8.586 10l-1.293 1.293a1 1 0 101.414 1.414L10 11.414l1.293
1.293a1 1 0 001.414-1.414L11.414 10l1.293-1.293a1 1 0 00-1.414-1.414L10 8.586 8.707
7.293z" clip-rule="evenodd"></path>
        </svg>

```

```

        <p class="text-xs font-medium">${data.message || 'Registration
failed'}</p>
    </div>
    `;
    }
    })
    .catch(err => {
        console.error(err);
        statusDiv.innerHTML = `
            <div class="text-red-600 text-center">
                <svg class="w-8 h-8 mx-auto mb-2" fill="currentColor" viewBox="0 0 20
20">
                    <path fill-rule="evenodd" d="M10 18a8 8 0 100-16 8 8 0 00 16zM8.707
7.293a1 1 0 00-1.414 1.414L8.586 10l-1.293 1.293a1 1 0 101.414 1.414L10 11.414l1.293
1.293a1 1 0 001.414-1.414L11.414 10l1.293-1.293a1 1 0 00-1.414-1.414L10 8.586 8.707
7.293z" clip-rule="evenodd"></path>
                </svg>
                <p class="text-xs font-medium">Connection error</p>
            </div>
        `;
    });
    });
</script>
</body>
</html>

```

detect_shirt.py

```

import cv2
import sys
import numpy as np
from ultralytics import YOLO
from collections import deque

# Load the YOLO model
model_path = "runs/content/runs/detect/train/weights/best.pt"

```

```

try:
    model = YOLO(model_path)
except Exception as e:
    print(f"Error: Could not load model. {e}")
    sys.exit(1)

camera_index = 0
for i in range(5):
    cap = cv2.VideoCapture(i)
    if cap.isOpened():
        print(f"Camera index {i} is available")
        camera_index = i
        cap.release()
        break

# Initialize video capture from the default webcam
cap = cv2.VideoCapture(camera_index)
if not cap.isOpened():
    print("Error: Could not open webcam.")
    sys.exit(1)

# Set a lower resolution to reduce camera lag
cap.set(cv2.CAP_PROP_FRAME_WIDTH, 640)
cap.set(cv2.CAP_PROP_FRAME_HEIGHT, 640)

# Define the reference color for the B.TECH department shirt (based on previous logs)
reference_color_rgb = np.array([148, 148, 146])

# Variables for frame skipping and temporal consistency
frame_counter = 0
FRAME_SKIP = 5 # Perform color detection every 5 frames
history = deque(maxlen=3) # Store the last 3 detection results for temporal consistency
last_detected_color = (0, 0, 0) # Store the last detected color for skipped frames
last_is_btech = False # Store the last detection result

```



```

# Function to calculate Euclidean distance between two RGB colors
def euclidean_distance(color1, color2):
    return np.sqrt(np.sum((color1 - color2) ** 2))

# Function to analyze shirt color using lightweight K-Means clustering
def analyze_shirt_color(frame, x1, y1, x2, y2, is_person_detection=False):
    # If the model detects "person" objects, focus on the upper half of the bounding box (shirt area)
    if is_person_detection:
        shirt_y1 = y1
        shirt_y2 = y1 + (y2 - y1) // 2
        shirt_x1 = x1
        shirt_x2 = x2
    else:
        shirt_y1 = y1
        shirt_y2 = y2
        shirt_x1 = x1
        shirt_x2 = x2

    # Ensure the region is valid
    shirt_y1 = max(shirt_y1, 0)
    shirt_y2 = min(shirt_y2, frame.shape[0])
    shirt_x1 = max(shirt_x1, 0)
    shirt_x2 = min(shirt_x2, frame.shape[1])

    if shirt_y2 <= shirt_y1 or shirt_x2 <= shirt_x1:
        return False, (0, 0, 0)

    # Extract the region of interest (ROI)
    roi = frame[shirt_y1:shirt_y2, shirt_x1:shirt_x2]
    if roi.size == 0:
        return False, (0, 0, 0)

```

```

# Downsample the ROI to reduce computation (resize to 32x32 pixels)
roi_resized = cv2.resize(roi, (32, 32), interpolation=cv2.INTER_AREA)

# Reshape the ROI into a 2D array of pixels for K-Means
pixels = roi_resized.reshape(-1, 3)
pixels = np.float32(pixels)

# Define criteria and apply K-Means clustering
criteria = (cv2.TERM_CRITERIA_EPS + cv2.TERM_CRITERIA_MAX_ITER, 50, 0.2)
k = 3
_, labels, centers = cv2.kmeans(pixels, k, None, criteria, 5,
cv2.KMEANS_RANDOM_CENTERS)

# Convert centers to integer
centers = np.uint8(centers)

# Find the dominant cluster (the one with the most pixels)
labels = labels.flatten()
dominant_cluster = np.argmax(np.bincount(labels))
dominant_color = centers[dominant_cluster]

# Convert dominant color to RGB for comparison
dominant_color_rgb = np.array([dominant_color[2], dominant_color[1],
dominant_color[0]])

# Convert dominant color to HSV to check saturation
dominant_color_bgr = np.uint8([[dominant_color]])
hsv_color = cv2.cvtColor(dominant_color_bgr, cv2.COLOR_BGR2HSV)
saturation = hsv_color[0][0][1]

# Calculate Euclidean distance between the dominant color and the reference color
distance = euclidean_distance(dominant_color_rgb, reference_color_rgb)

# Check if the color is "greyish" (low saturation) and close to the reference color

```

```

# Saturation threshold: 20 (about 8% in OpenCV's 0-255 scale)
# Distance threshold: 12 (stricter for better accuracy)
is_btech = (distance < 12) and (saturation < 20)

# Return the dominant color in BGR for display
avg_color = tuple(int(c) for c in dominant_color)

return is_btech, avg_color

```

try:

```

while True:
    # Capture frame
    ret, frame = cap.read()
    if not ret:
        print("Error: Could not read frame.")
        break

    # Initialize a default color for the square
    detected_color = last_detected_color
    is_btech = last_is_btech

    # Perform inference using the YOLO model
    results = model(frame)

    # Increment frame counter
    frame_counter += 1

    # Process detections
    for result in results:
        boxes = result.boxes
        for box in boxes:
            # Extract coordinates and class
            x1, y1, x2, y2 = map(int, box.xyxy[0])
            class_id = int(box.cls)

```

```

class_name = model.names[class_id]

# Determine if the detection is a person or a shirt
is_person_detection = class_name.lower() == "person"

# Skip if the class is neither "shirt" nor "person"
if not (class_name.lower() in ["shirt", "person"]):
    continue

# Perform color detection only every FRAME_SKIP frames
if frame_counter % FRAME_SKIP == 0:
    # Analyze the shirt color using lightweight K-Means
    is_btech_temp, detected_color_temp = analyze_shirt_color(frame, x1, y1, x2, y2,
is_person_detection)

    # Update the last detected values
    last_detected_color = detected_color_temp
    detected_color = detected_color_temp

    # Add the detection result to history for temporal consistency
    history.append(is_btech_temp)

    # Check if the shirt is consistently detected as B.TECH over the last few frames
    if len(history) == history.maxlen:
        is_btech = all(history) # True only if all recent detections are B.TECH
    else:
        is_btech = False # Default to False until history is full

    last_is_btech = is_btech

    # Print the detected color in RGB format
    print(f"Detected shirt color (RGB): ({detected_color[2]}, {detected_color[1]},
{detected_color[0]})")

```

```

# Draw rectangle (red, thickness=2)
cv2.rectangle(frame, (x1, y1), (x2, y2), (0, 0, 255), 2)

# Determine the text to display based on shirt color
text = "B.TECH" if is_btech else "Unknown"

# Draw a background rectangle for better text visibility
(text_width, text_height), _ = cv2.getTextSize(text,
cv2.FONT_HERSHEY_SIMPLEX, 0.5, 2)
rect_x1, rect_y1 = x1, y1 - text_height - 5
rect_x2, rect_y2 = x1 + text_width, y1 - 5
cv2.rectangle(frame, (rect_x1, rect_y1), (rect_x2, rect_y2), (0, 0, 255), -1)
cv2.putText(frame, text, (x1, y1 - 5),
            cv2.FONT_HERSHEY_SIMPLEX, 0.5, (0, 0, 0), 2)

# Draw a filled square in the bottom-right corner to show the detected color
square_size = 50
margin = 10
frame_height, frame_width = frame.shape[:2]
square_x1 = frame_width - square_size - margin
square_y1 = frame_height - square_size - margin
square_x2 = frame_width - margin
square_y2 = frame_height - margin

# Ensure detected_color is a valid tuple of integers
if not isinstance(detected_color, tuple) or len(detected_color) != 3 or not all(isinstance(c,
(int, np.integer)) for c in detected_color):
    detected_color = (0, 0, 0)

# Draw the filled square with the detected color
cv2.rectangle(frame, (square_x1, square_y1), (square_x2, square_y2), detected_color,
-1)
cv2.rectangle(frame, (square_x1, square_y1), (square_x2, square_y2), (0, 0, 0), 1)

# Display the frame

```

```
cv2.imshow('Shirt Detection', frame)

# Break on 'q' key press
if cv2.waitKey(1) & 0xFF == ord('q'):
    break

finally:
    # Cleanup
    cap.release()
    cv2.destroyAllWindows()
```